

FLAGSHIP BUILD SPECIFICATION

[BRAND]

*The single source of truth for every part of the business.
Architecture, data models, customer journey, fulfillment,
marketing, compliance, security — and every question to ask the brand partner.*

Document	Flagship Build Specification v1.0
Project	Luxury Pickleball Jewelry E-Commerce
Brand Partner	Shah Luxury
Audience	Project owner (build reference) + brand partner (vendor inputs)
Engineering stack	Next.js 16 · React 19 · TypeScript · Prisma · Turso · Stripe
Hosting	Vercel
Purpose	Complete operational blueprint — data, code, workflows, vendor specs
Document length	Ten volumes + appendices

DOCUMENT MAP

Contents

How this document is organized

This specification is structured as ten Volumes plus appendices. Volumes 1–4 cover what to build. Volumes 5–8 cover how to run it. Volume 9 is the consolidated vendor request to send the brand partner. Volume 10 is the launch and business operations plan. Appendices contain reference material — schemas, API contracts, environment variables, and templates.

BUILD — Foundation, Data, Customer Journey, Operations

Volume 1	Strategic Foundation & Tech Stack	5
Volume 2	Data Model & Domain Logic	10
Volume 3	Customer Journey & UX Specification	22
Volume 4	Fulfillment, Shipping & Operations	32

RUN — Marketing, Compliance, Security, Performance

Volume 5	Marketing, SEO & Customer Retention	42
Volume 6	Legal, Compliance & Trust	48
Volume 7	Security, Authentication & Audit	54
Volume 8	Performance, Caching & Scale	60

PARTNER & PLAN

Volume 9	Brand Partner Specification & Requests	64
Volume 10	Business Operations & Launch Plan	74

APPENDICES

Appendix A	Full Prisma Schema (current + recommended)	80
Appendix B	API Endpoint Reference	84
Appendix C	Environment Variables Checklist	86
Appendix D	Order Status State Machine	88
Appendix E	Email Template Library	89
Appendix F	Vendor & Tooling Comparison Matrix	91
Appendix G	Glossary	93

OVERVIEW

Executive Summary

[BRAND] is a direct-to-consumer luxury jewelry e-commerce business selling handcrafted pickleball-inspired pieces in partnership with Shah Luxury. This document specifies everything required to build, launch, fulfill, and operate the site at flagship quality.

The site is technically built on Next.js 16 / React 19 with a Prisma data layer, custom JWT authentication, Stripe Checkout for payments, and a configurator covering three silhouettes (Pickleball, Paddle, Court). The security posture is solid — Argon2id passwords, refresh-token family rotation, audit logging, encrypted sensitive fields. What separates the current state from a true flagship is not engineering — it is **operational completeness**: production data store, fulfillment workflow, post-purchase customer experience, marketing automation, and locked-down vendor relationships.

What this document delivers

- A complete data model covering orders, fulfillment, returns, wishlists, saved configurations, reviews, discount codes — including a recommended Prisma schema (Appendix A).
- API contracts for every endpoint the site needs (Appendix B).
- An order lifecycle state machine that drives every backend operation (Appendix D).
- An end-to-end customer journey spec — discovery, product detail, configurator, cart, checkout, account, returns.
- A fulfillment runbook covering carrier integration, address validation, international shipping, returns, and warranty repairs.
- Marketing stack — email automation, reviews, SEO, structured data.
- Legal & compliance — GDPR, CCPA, FTC Jewelry Guides, PCI scope, cookie consent.
- Security model — authentication, authorization, audit logging, secrets management, incident response.
- Performance & scale — caching, CDN, image strategy, observability.
- A consolidated brand partner request — ~80 questions and information needs for Shah Luxury.
- Realistic cost structure, ongoing operating cadence, 90-day launch plan.

Reading order

- **Building from scratch?** Read straight through. Volumes 1–4 build the site, 5–8 operate it.
- **Discussing with the brand partner?** Send Volume 9 alone — it stands as a complete request document.
- **Looking up a specific thing?** Use the table of contents and the appendices — they're written to be referenced in isolation.
- **Estimating effort?** See Volume 10 for the cost summary and roadmap.

The single largest open risk

Pricing — must be answered before live mode

All catalog and customize-page prices are currently // TODO placeholders. The checkout API recomputes every line item server-side from these values — meaning whatever sits in the code is what the customer pays. Until Shah Luxury supplies authoritative wholesale prices and the MSRP policy, the site cannot accept real orders without commercial risk. See §4.1 in Volume 9.

VOLUME 1

Strategic Foundation & Tech Stack

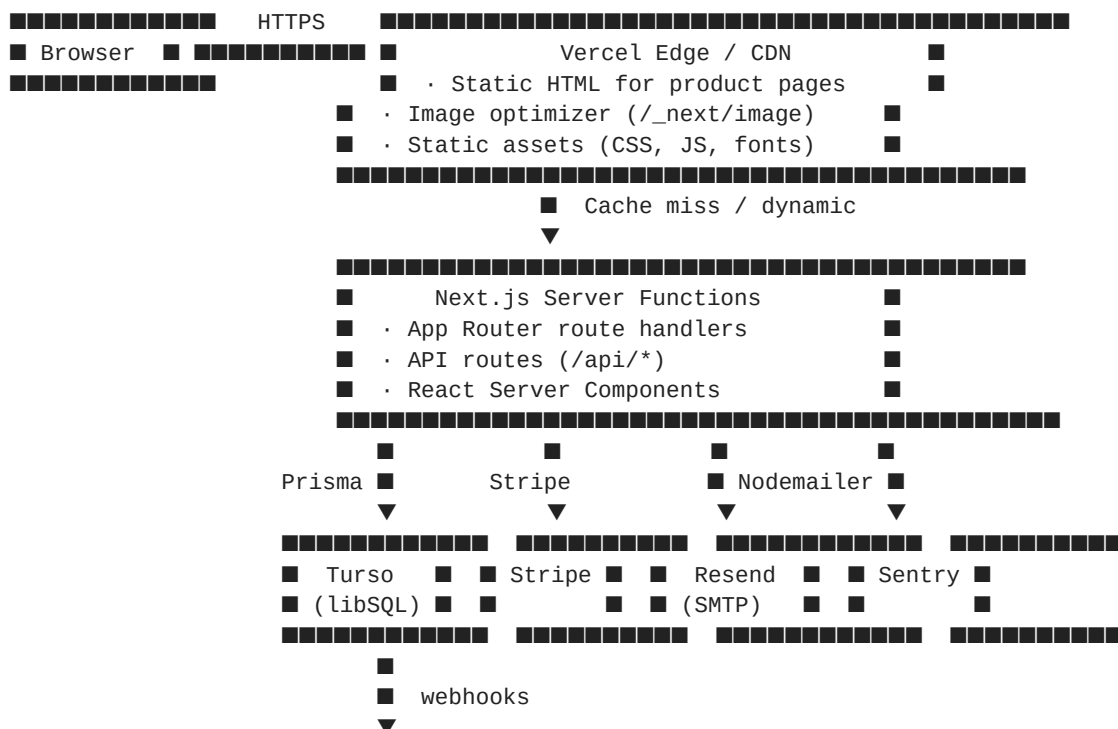
1.1 What does “flagship” mean for [BRAND]?

A flagship e-commerce site is not just operational — it is the brand's most important physical asset in the digital world. The bar is higher than a typical Shopify store across five dimensions:

- **Editorial-quality presentation** — typography, photography, motion, and pacing communicate luxury before the customer even understands the price.
- **Trust through transparency** — pricing, materials, lead times, warranty, and returns are stated plainly. Hidden fees and vague policies are the surest signs of an unserious brand.
- **Operational reliability** — every order lands, every email arrives, every tracking number works, every customer service interaction is on brand.
- **Data ownership and respect** — customer information is collected with consent, encrypted at rest, and never sold. GDPR & CCPA aren't compliance burdens, they are alignment with brand values.
- **Engineering discipline** — the codebase can be operated by one person for years without becoming a liability. Boring choices in the boring places; investment where it shows.

1.2 Architecture at a glance

[BRAND] runs on a modern serverless architecture deployed to Vercel. The browser hits Vercel's edge network, which serves prerendered HTML directly when possible and invokes Next.js server functions when dynamic. Server functions read from a managed libSQL database (Turso), call Stripe for payments, send mail via a transactional provider, and write structured logs to Sentry.



[illegible]

1.3 Tech stack reference

Layer	Choice	Why this choice
Web framework	Next.js 16 App Router	Server components, streaming, file-system routing, built-in image optimization, first-class Vercel deployment. The same stack powers Notion, Cal.com, Loom.
UI library	React 19	Server Components reduce bundle size; Suspense streaming improves perceived speed.
Styling	Tailwind CSS 4 + CSS variables	Atomic utilities for layout; CSS variables for the design system tokens (color, type scale, spacing).
Type system	TypeScript 5 strict mode	Catches whole categories of error at edit time. Mandatory at this scale.
Data access	Prisma 7 with libSQL adapter	Type-safe queries from the schema; same code in dev (SQLite) and prod (Turso).
Database	Turso (managed libSQL)	Sub-second cold-start, replication built in, identical adapter to local SQLite. Postgres is the alternative — both work; Turso is simpler here.
Authentication	Custom JWT + Argon2id + refresh family	Industry-standard primitives; gives full control over session policies. Clerk / Auth.js are valid alternatives if you want to outsource.
Payments	Stripe Checkout + webhooks	PCI scope stays at SAQ-A (the easy one); no card data touches our servers.
Email	Nodemailer over SMTP (Resend / Postmark)	Standard interface, swap providers via env var. Avoid Gmail SMTP for deliverability.
Error monitoring	Sentry	Captures both server and client errors with stack traces.
Rate limiting	Upstash Redis (recommended)	Per-process counters fail on serverless; needs a shared store.
Image hosting	Vercel image optimization	Built into hosting; serves WebP/AVIF with responsive srcsets.
3D / motion	Three.js + R3F + GSAP + Lenis + Framer Motion	Selectively used. R3F lazy-loaded so 3D-free routes do not pay the bundle cost.
Search (later)	Algolia or Postgres full-text	Not needed at <100 SKUs.
Analytics	Plausible (or PostHog) + Vercel Analytics	Privacy-friendly product analytics; Vercel for Core Web Vitals.

1.4 Project directory structure

The repository is laid out so any single concern can be found from its name:

src/	
app/	Next.js App Router
(marketing)/	Public pages (homepage, story, concierge)
api/	Server routes
auth/	login, register, refresh, logout, reset
checkout/	POST creates Stripe session
webhook/	POST receives Stripe events
admin/	RBAC-gated read endpoints
account/	Customer self-service endpoints (planned)
cart/	Bag page
checkout/	Stripe-redirected pages
collections/	Product listing
customize/	Configurator + bespoke scrub-video pages
product/[id]/	Detail page
profile/	Authenticated account UI
login/	Auth UI
layout.tsx	Root layout: fonts, providers, head
components/	
configurator/	Configurator UI (Option, Preview, Spec, Price)
customize/	Shared ScrubVideo
layout/	Navigation, Footer
product/	ProductDetailView, gallery
sections/	Homepage sections, hero, brand story
three/	R3F components (lazy-loadable)
ui/	Buttons, badges, animated images, primitives
lib/	
catalog.ts	Catalog config (pieces, axes, prices)
catalog-axes.ts	Axis resolver
pricing.ts	Estimation engine
products.ts	Static product catalog
cart-context.tsx	Cart provider (localStorage-backed)
auth-context.tsx	Auth provider (in-memory + refresh cookie)
rate-limit.ts	(To migrate to Upstash)
customize.ts	Bespoke route registry
server/	Server-only modules
auth.ts	Password, JWT, refresh, sessions, crypto
audit.ts	Audit log, IP block, RBAC
db.ts	Prisma client singleton
email.ts	Transactional templates
stripe.ts	Stripe SDK, checkout session, webhook construct
price.ts	Server-side price recompute (truth)
middleware.ts	withAuth, withRateLimit, withWAF, withCSRF
request.ts	getClientIp helper
middleware.ts	Edge middleware (CSRF, null-byte WAF)
prisma/	
schema.prisma	Database schema
public/	
images/	Site + product photography
images/catalog/	Configurator variant photos
videos/	Scrub videos for bespoke pages

1.5 Engineering principles

- **Server is the source of truth.** Prices, permissions, business rules are recomputed on the server. The client is convenience UI.

- **Boring is good.** Stripe for payments, Postgres-like database, off-the-shelf SaaS. Custom code only where it improves the product.
- **Reversibility.** Every deploy can be rolled back in under a minute. Every database migration is reversible or has a documented rollback procedure.
- **Observability before optimization.** Sentry first. Performance tuning only with metrics in hand.
- **Secrets fail closed.** Missing production secrets throw at first use, never fall back to dev defaults.
- **Audit everything that matters.** Login, registration, checkout, admin reads, order state changes — all written to the AuditLog table with IP, user agent, and outcome.

VOLUME 2

Data Model & Domain Logic

The data model is the spine of the business. Every workflow — checkout, fulfillment, returns, customer support, analytics — reads or writes one of these entities. This volume specifies the complete schema needed for flagship operation.

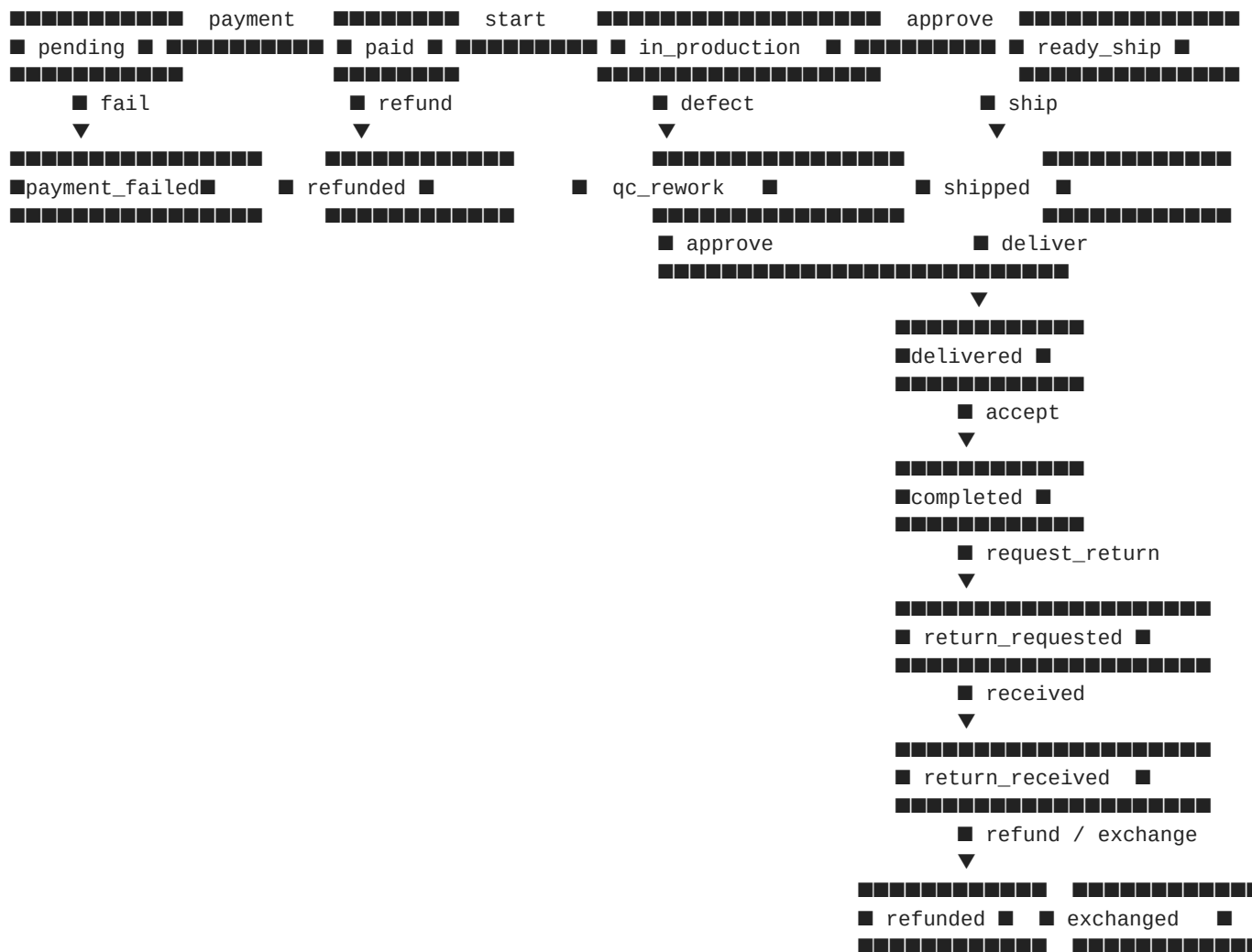
2.1 Entity overview

Entity	Status today	Purpose
User	Built	Account, role, lockout state, MFA fields (unused).
Session	Built	Server-validated session token, IP/UA tracked.
RefreshToken	Built	Family-based rotation, used flag for reuse detection.
PasswordReset	Built	One-hour reset tokens, hashed at rest.
Order	Built — extend	Add carrier, tracking number, production timestamps, internal notes.
OrderItem	Built	Line items, denormalized productName and unit price.
OrderEvent	Add	Append-only audit of state transitions per order.
Address	Built — extend UI	Saved customer addresses; needs UI surface.
AuditLog	Built	Security-relevant event log.
BlockedIP	Built	IP blacklist with expiry.
ReturnRequest	Add	RMA, reason, status, refund linkage.
Coupon	Add	Discount codes, usage cap, expiry.
Wishlist / WishlistItem	Add	Saved pieces for later purchase or sharing.
SavedConfiguration	Add	Persist configurator state for return / share.
ProductReview	Add	UGC reviews on product detail pages.
EmailSubscription	Add	Newsletter + back-in-stock alerts.
WebhookEvent	Add	Idempotency log for Stripe webhook events.

2.2 Order lifecycle state machine

Every order moves through a defined sequence of states. The state machine below is the canonical definition. Backend code MUST validate that any status transition is allowed by this graph, and MUST write

an OrderEvent row for every transition.



Allowed transitions (TypeScript):

```

export const ORDER_STATUSES = [
  'pending', 'payment_failed',
  'paid', 'in_production', 'qc_rework', 'ready_ship',
  'shipped', 'out_for_delivery', 'delivered', 'completed',
  'cancelled', 'refunded',
  'return_requested', 'return_received', 'exchanged',
] as const;
export type OrderStatus = typeof ORDER_STATUSES[number];

export const ALLOWED_TRANSITIONS: Record<OrderStatus, OrderStatus[]> = {
  pending: ['paid', 'payment_failed', 'cancelled'],
  payment_failed: ['pending', 'cancelled'],
  paid: ['in_production', 'cancelled', 'refunded'],
  in_production: ['ready_ship', 'qc_rework', 'cancelled'],
  qc_rework: ['in_production', 'ready_ship'],
  ready_ship: ['shipped'],
  shipped: ['out_for_delivery', 'delivered'],
  out_for_delivery: ['delivered'],
  delivered: ['completed', 'return_requested'],
  completed: ['return_requested'],
  return_requested: ['return_received', 'cancelled'],
  return_received: ['refunded', 'exchanged'],
  cancelled: [],
}

```

```

    refunded:      [],
    exchanged:     [],
  };

export function canTransition(from: OrderStatus, to: OrderStatus): boolean {
  return ALLOWED_TRANSITIONS[from]?.includes(to) ?? false;
}

```

Implementation note

The Order.status column is a TEXT field today. Keep it text — Prisma does not support proper enums across all databases — but enforce the union type at the application layer via the constant above. Every API or admin action that changes status must call `canTransition()` first and write a row to OrderEvent.

2.3 Recommended Prisma schema extensions

Below are the new models and field additions required to operate the site at flagship quality. These are **additive** — they don't break anything currently shipped. Each is grouped by concern.

Order — operational fields

```

model Order {
  id              String      @id @default(uuid())
  userId          String
  status          String      @default("pending")
  totalCents      Int
  subtotalCents   Int          // new – pre-tax, pre-shipping
  taxCents        Int          @default(0)
  shippingCents   Int          @default(0)
  discountCents   Int          @default(0)
  currency        String      @default("usd")

  // Stripe
  stripePaymentId String?     @unique
  stripeSessionId String?     @unique

  // Shipping
  shippingAddress String?     // JSON
  billingAddress   String?     // JSON
  carrier          String?     // new – "fedex" | "ups" | "usps" | ...
  trackingNumber   String?     // new
  trackingUrl      String?     // new

  // Lifecycle timestamps
  paidAt           DateTime?
  productionStartedAt DateTime?
  shippedAt         DateTime?
  deliveredAt      DateTime?
  completedAt      DateTime?
  cancelledAt      DateTime?
  refundedAt       DateTime?

  // Internal
  notes            String?     // internal CS notes
  giftMessage      String?
  couponCode       String?
  createdAt        DateTime    @default(now())
}

```

```

    updatedAt          DateTime @updatedAt

    user               User      @relation(fields: [userId], references: [id])
    items              OrderItem[]
    events             OrderEvent[]
    returnRequests     ReturnRequest[]

    @@index([userId])
    @@index([stripePaymentId])
    @@index([status, createdAt])
    @@index([trackingNumber])
  }

```

OrderEvent — append-only audit of state transitions

```

model OrderEvent {
  id          String  @id @default(uuid())
  orderId     String
  fromState   String?
  toState     String
  actor       String  // "system" | "user" | "stripe" | "admin:<id>"
  reason      String?
  metadata    String? // JSON for tracking number changes, refund amounts, etc.
  createdAt   DateTime @default(now())

  order Order @relation(fields: [orderId], references: [id], onDelete: Cascade)

  @@index([orderId, createdAt])
}

```

ReturnRequest — RMA workflow

```

model ReturnRequest {
  id          String  @id @default(uuid())
  orderId     String
  rmaCode     String  @unique           // human-readable, e.g. "RMA-2026-0001"
  status      String  @default("requested")
  // requested → label_sent → in_transit → received → inspected → refunded / exchanged / denied
  reason      String                               // dropdown value
  reasonDetail String?                             // freeform from customer
  itemIds     String                               // JSON array of OrderItem ids
  refundCents Int?
  exchangeOrderId String?                         // if exchanged for a different piece
  returnLabelUrl String?
  internalNotes String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  order Order @relation(fields: [orderId], references: [id], onDelete: Cascade)

  @@index([orderId])
  @@index([status])
  @@index([rmaCode])
}

```

Coupon — discount codes

```

model Coupon {
  id          String  @id @default(uuid())
  code        String  @unique
  description  String?
  discountType String  // "percent" | "fixed"
}

```

```

    amount          Int                // percent (1-100) or cents
    minOrderCents    Int?
    maxUses          Int?
    usedCount        Int                @default(0)
    perUserLimit     Int?
    validFrom        DateTime?
    validUntil       DateTime?
    active           Boolean @default(true)
    createdAt        DateTime @default(now())

    @@index([code])
    @@index([active, validUntil])
}

```

Wishlist + WishlistItem

```

model Wishlist {
  id          String  @id @default(uuid())
  userId      String  @unique
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  user User          @relation(fields: [userId], references: [id], onDelete: Cascade)
  items WishlistItem[]
}

model WishlistItem {
  id          String  @id @default(uuid())
  wishlistId  String
  productId   String                // matches catalog id or configurator-derived id
  customization String?             // JSON snapshot of configurator state
  notes       String?
  addedAt     DateTime @default(now())

  wishlist Wishlist @relation(fields: [wishlistId], references: [id], onDelete: Cascade)

  @@unique([wishlistId, productId, customization])
}

```

SavedConfiguration — share/return to a configurator build

```

model SavedConfiguration {
  id          String  @id @default(uuid())
  userId      String? // null if anonymous (shareable URL only)
  pieceId     String  // catalog piece id
  selection   String  // JSON of the full sel object
  shareSlug   String  @unique // short URL-safe id used in /c/<slug>
  priceCents  Int      // snapshot at save time
  createdAt   DateTime @default(now())

  @@index([userId])
  @@index([shareSlug])
}

```

ProductReview — UGC on product detail

```

model ProductReview {
  id          String  @id @default(uuid())
  productId   String  // matches catalog id
  userId      String  // verified owner of the order containing this product
  orderId     String? // optional verification of purchase
  rating      Int      // 1-5
}

```

```

    title      String?
    body       String?
    verified   Boolean @default(false)
    hidden     Boolean @default(false)
    createdAt  DateTime @default(now())

    user User @relation(fields: [userId], references: [id], onDelete: Cascade)

    @@index([productId, hidden, createdAt])
    @@index([userId])
  }

```

EmailSubscription — newsletter + back-in-stock alerts

```

model EmailSubscription {
  id          String  @id @default(uuid())
  email       String  @unique
  userId      String?
  source      String  // "footer" | "checkout" | "back_in_stock:<sku>"
  productIds  String? // JSON array for back-in-stock specifics
  confirmed   Boolean @default(false)
  confirmedAt DateTime?
  unsubscribed Boolean @default(false)
  unsubAt     DateTime?
  createdAt   DateTime @default(now())

  @@index([email])
  @@index([unsubscribed, confirmed])
}

```

WebhookEvent — Stripe idempotency record

```

model WebhookEvent {
  id          String  @id @default(uuid())
  provider     String  // "stripe"
  eventId      String  @unique // Stripe event id, used for dedupe
  type         String  // event.type
  payloadHash  String  // SHA-256 of body for verification
  processedAt  DateTime @default(now())

  @@index([provider, type])
}

```

2.4 Pricing engine — the single source of truth

Every line item in a checkout is re-priced server-side. The client price is treated as a hint to surface drift, not as authoritative. The resolver covers three id formats.

```

/**
 * Returns the canonical price in cents for any productId the app produces.
 * Returns null if the id is unknown – callers reject the checkout.
 *
 * Id formats:
 * 1. Static product:      "court-royale-pendant"
 * 2. Bespoke variant:    "sparkle-studs-silver__twl-050"
 * 3. Configurator derived: "ball-pendant__metalColor-yellow_treatment-plain_karat-14k..."
 */
export function resolvePriceCents(productId: string): number | null {
  // ... see src/lib/server/price.ts
}

```

Pricing formula for configurator pieces

```
estimatePrice(piece, sel) =
  piece.basePrice
  × karat[sel.karat].multiplier
  + metalColor[sel.metalColor].priceAdd
  + accent[sel.accent].priceAdd
  + (treatment ≠ 'plain' ? sel.caratVersion × RATE : 0)
  × cutGrade[sel.cutGrade].multiplier      (if has stones)
  × clarity[sel.clarity].multiplier          (if has stones)
  + ballAccent[sel.ballAccent].priceAdd
  + chainLength[sel.chainLength].priceAdd
```

Required from Shah Luxury — see Volume 9 §9.1

Every constant in this formula — basePrice per piece, karat multipliers, the per-carat RATE, cut and clarity multipliers, surcharges per metal/accent/chain/ball-accent — must be supplied authoritatively. Without these numbers from the brand partner the formula yields placeholders, and customers are billed those placeholders.

2.5 Catalog configuration model

Products are not Prisma rows. They are static data declared in TypeScript so the catalog config is type-checked, prerendered, and version-controlled. The configurator reads this same model to render only the option axes each piece actually supports.

```
export interface Piece {
  id: string; // url + image path slug
  name: string;
  codes: string[]; // SKUs this piece consolidates
  form: 'ball' | 'paddle' | 'court';
  wearType: 'pendant' | 'earring' | 'necklace';
  siteProductId: string | null; // mapping to static product entry
  basePrice: number;
  axes: {
    metalColor?: string[];
    karat?: string[];
    treatment?: string[]; // plain | gypsy | pave | baguette | outline
    accent?: string[]; // diamond | emerald | enamel
    enamelColor?: string[];
    earringBack?: string[];
    ballAccent?: string[];
    orientation?: string[]; // horizontal | vertical
    chainLength?: number[];
    caratVersion?: number[];
    cutGrade?: string[];
    clarity?: string[];
  };
  defaults: Record<string, string | number>;
  video?: string | null;
  imageBase: string;
  notes?: string;
}
```

Why this is in code, not the database: a catalog change is a code change. It gets reviewed, built, and rolled back like any other deploy. Prices in code are surfaced in the Git diff before they go live.

2.6 User model and role-based access

Users are stored in Prisma with role-based access (RBAC). Four roles exist; permission maps live in `src/lib/server/audit.ts`.

Role	Capabilities
guest	Browse catalog, read product details. No checkout.
user	Browse + checkout, manage own profile / orders / wishlist.
admin	Read users + orders, modify order status, issue refunds.
superadmin	All admin powers + manage roles, view audit logs.

VOLUME 3

Customer Journey & UX Specification

This volume defines every customer-facing surface — what's rendered, what data backs it, what the user can do on it, and how it connects to the next step. Each section names the route, the page-level data it needs, and the user actions it supports.

3.1 Discovery — homepage, collections, brand story

The discovery surface is the editorial entry point. Conversion is not the explicit goal — brand alignment and lead capture are. Real purchase intent comes later, on the configurator or the product detail page.

Route	Purpose	Key elements
/	Brand hero, story teaser, three collection cards, concierge teaser	Parallax hero, GSAP SplitText reveal, style quiz CTA, editorial divider
/collections	All three tiers (Victory, Signature, Legacy)	CollectionHero per tier; CollectionGrid; complementary cross-sell
/story	Brand origin, craftsmanship film	Editorial layout; scrub-style video; community testimonial grid
/concierge	White-glove consultation booking	Service cards (virtual consultation, sizing kit, bespoke); form

Discovery — what's required from Shah Luxury

- Hero photography (or video) of jewelry in luxury editorial context.
- Brand origin text and craftsmanship narrative.
- Lifestyle imagery covering at least 8 community-feel shots for the story page.
- Approved press mentions if any.
- Approval to commission a styled photoshoot using their pieces.

3.2 Product detail page

Route: /product/[id]. Server-rendered with full metadata (SEO, OG, JSON-LD). Five sections, top to bottom.

Section	Content	Backed by
1. Gallery (left ~55%)	Main image with prev/next arrows, swipe support on mobile, thumbnail strip below.	product.gallery[] — array of {src, alt, label}

Section	Content	Backed by
2. Detail panel (right ~45%, sticky)	Collection eyebrow, product name, SKU code, tagline, price, divider, description, spec table, Add to Bag button, Customize CTA (if mapped), reassurance line.	product.specs, lib/customize.CUSTOMIZE_ROUTES
3. Reviews	Aggregate rating + recent reviews. Verified-purchase badge for owners.	ProductReview
4. Complementary products	Cross-sell carousel of related pieces.	product.complementaryIds
5. JSON-LD	Structured data for Google Shopping / rich snippets.	Computed from product

JSON-LD payload to ship on every product page

```
{
  "@context": "https://schema.org",
  "@type": "Product",
  "name": product.name,
  "image": product.gallery.map(g => g.src),
  "description": product.description,
  "sku": product.productCode,
  "brand": { "@type": "Brand", "name": "[BRAND]" },
  "offers": {
    "@type": "Offer",
    "url": `https://[domain]/product/${product.id}`,
    "priceCurrency": "USD",
    "price": product.price.toFixed(2),
    "availability": "https://schema.org/MadeToOrder",
    "itemCondition": "https://schema.org/NewCondition",
    "seller": { "@type": "Organization", "name": "[BRAND]" }
  },
  "aggregateRating": reviews.count > 0 ? {
    "@type": "AggregateRating",
    "ratingValue": reviews.average,
    "reviewCount": reviews.count
  } : undefined
}
```

3.3 Configurator

Routes: /customize (silhouette index + generic configurator) and /customize/[pieceId] (per-piece direct link). Three bespoke pieces have dedicated scrub-video routes at /customize/sparkle-studs-silver, /customize/sparkle-drops-silver, and /customize/paddle-pave-silver.

Generic configurator flow (left → right)

Step	User action	Outcome
1	Pick silhouette (Pickleball / Paddle / Court)	Style picker repopulates with that family's pieces.
2	Pick style (e.g. Pickleball Earrings)	Design + Specifications panels render only the axes that piece declares.

Step	User action	Outcome
3	Pick Design axes (metal, finish, diamond weight, etc.)	Live preview updates; price recomputes; spec sheet rewrites.
4	Pick Specifications axes (karat, cut, clarity, chain)	Same — but informational axes don't change the photo.
5	Begin Your Commission	Synthesizes a derived Product with a unique id, adds to cart.

Key invariant

The configurator never sends the user's price to the server in a privileged way. The cart line's `priceInCents` is recomputed at checkout via `resolvePriceCents()` — the client is just convenience UI.

3.4 Cart / Shopping Bag

Route: `/cart`. State lives in a React context backed by `localStorage` (`courset-cart`) so the cart survives reloads without a database round-trip for anonymous users.

State key	Semantics
<code>items[]</code>	Array of <code>{product, quantity, customization?}</code> . product.id is the cart key — derived ids (with <code>__variant</code>) give each configuration its own line.
<code>totalItems</code>	Sum of quantities.
<code>totalPrice</code>	Client-side sum of <code>product.price × quantity</code> . Not authoritative — server recomputes on checkout.

Empty state and populated state

- Empty: serene hero copy, primary CTA to `/collections`, secondary CTA to `/customize`.
- Populated: line items with image, name, customization summary, qty +/-, remove. Sticky order summary sidebar: subtotal, free shipping line, total. Checkout button routes to `/login` if unauthenticated, otherwise calls `POST /api/checkout`.

3.5 Checkout

Server route: `POST /api/checkout`. The handler validates the cart (Zod), recomputes every price (`lib/server/price`), creates a Stripe Checkout Session, and returns a redirect URL. Stripe hosts the actual card-entry surface — keeps PCI scope at SAQ-A.

Server-side checkout pipeline

```
POST /api/checkout
■
■■■ getClientIp(req)
■■■ rateLimit(`checkout:${ip}`, 5, 60_000)
■■■ verifyAccessToken(authHeader) → 401 if invalid
```

```

■■■ CheckoutSchema.safeParse(body)      → 400 if malformed
■■■ prisma.user.findUnique              → 404 if user gone
■
■■■ for each item:
■   resolvePriceCents(item.productId) → 400 if unknown product
■   log price-drift to audit
■   sanitize item with server price
■
■■■ createCheckoutSession(items, user)
■   · line items use SANITIZED prices
■   · shipping_address_collection: US/CA/GB/AU
■   · metadata.userId = user.id
■   · success_url / cancel_url
■
■■■ logAudit('checkout.create', success)
■■■ 200 { url }

```

Address validation (recommended addition)

Stripe Checkout collects shipping addresses but doesn't verify they're deliverable. For pieces above \$500, a typo can cost the brand the piece. Two options:

- **Google Places autocomplete** on the Stripe Checkout page — Stripe supports this natively via the dashboard. Free.
- **Smarty** (\$30/mo for 1k validations) or **Lob** (pay per call) — server-side validation called from the Stripe webhook on `checkout.session.completed`. If address is invalid, transition the order to `address_review` and email the customer to confirm before production starts.

Tax calculation

- Enable **Stripe Tax** in the dashboard. Cost: 0.5% of the tax amount.
- Register for sales-tax permits in every state where you have nexus (physical, economic, or affiliate). TaxJar (\$19–\$99/mo) automates state filings.
- EU customers — register for IOSS (Import One Stop Shop) for orders under €150, otherwise the customer pays VAT + customs at delivery and bounces.

3.6 Account & profile (authenticated)

Route: `/profile`. Protected — redirects to `/login` if no user. Today shows placeholder dashboard tiles. Below is the target specification.

Tile	Lives at	Powered by
Recent orders	<code>/profile</code> top section	GET <code>/api/account/orders</code> (planned)
Order detail	<code>/profile/orders/[id]</code>	GET <code>/api/account/orders/[id]</code> (planned)
Saved addresses	<code>/profile/addresses</code>	GET/POST/PUT/DELETE <code>/api/account/addresses</code> (planned)
Wishlist	<code>/profile/wishlist</code>	GET/POST/DELETE <code>/api/account/wishlist</code> (planned)

Tile	Lives at	Powered by
Saved configurations	/profile/configurations	SavedConfiguration model
Settings (password, email, MFA)	/profile/settings	PUT /api/account/profile, POST /api/auth/mfa
GDPR data export	/profile/data	POST /api/account/export (planned)
Account deletion	Settings page modal	DELETE /api/account (planned)

3.7 Order detail page (customer view)

Route: /profile/orders/[id]. The customer's view of one order across its lifecycle.

Section	Shown when	Content
Status banner	Always	Big status chip: Processing · In Production · Shipped · Delivered · Returned
Production timeline	status ≥ paid	Visual timeline showing paid → in production → shipped → delivered with timestamps
Tracking	status ≥ shipped	Carrier name, tracking number with a link to the carrier's tracking site
Items	Always	Each line item: image, name, customization summary, qty, line total
Totals	Always	Subtotal, tax, shipping, discount, total
Shipping address	Always	Read-only display
Actions	Conditional	Cancel (if status = pending/paid), Request return (if status = delivered/completed), Contact concierge (always), Download receipt

3.8 Wishlist

- Add to wishlist from product detail and configurator preview (heart icon).
- Anonymous users get a localStorage-backed list that merges into their account on first sign-in.
- Wishlist page shows added pieces with current price; re-prices on view in case catalog changed.
- Each item can be shared via short URL (uses SavedConfiguration model for configurator items).
- **Back-in-stock alerts** — even though the catalog is largely made-to-order, this matters for specific stones the brand sometimes runs out of.

3.9 Search (deferred to scale)

Not needed for <100 SKUs. Browse-by-attribute (the configurator filters) is enough. Add Algolia (\$50/mo) or Postgres full-text search when the catalog exceeds 100 pieces or when customer support data shows people typing search queries the navigation can't answer.

VOLUME 4

Fulfillment, Shipping & Operations

This is the volume that turns a website into a business. Receiving an order is one thing; actually getting a \$1,500 piece to a customer's door in flawless condition is another. Three things must be true: the order data is right, the operational hand-off to the brand partner is reliable, and the customer is informed at every stage.

4.1 Drop-ship vs. self-ship — pick one model

The choice has consequences across customer experience, branding, inventory, and the codebase. Pick one and build for it; do not try to support both day one.

Model	Pros	Cons	Code impact
Drop-ship Shah ships direct to customer	No warehousing, no shipping labels, no inventory; lowest [BRAND] operational burden.	Less control over packaging/branding; depends on Shah's lead times; harder to differentiate.	Minimal — need a hook on order-paid to email/notify Shah with shipping address & SKU. Tracking number arrives by reply email or via a shared spreadsheet.
Self-ship Shah ships to [BRAND], [BRAND] ships to customer	Full control over branded packaging, insert cards, polishing cloth; can add a personal touch; QC pass before shipping.	Need a workspace, packaging materials, shipping software, label printer; higher per-order time cost.	Substantial — ShipStation/Shippo integration, label generation, internal admin UI for “Mark as shipped”.
Recommendation for launch Start with drop-ship . It removes 80% of the operational burden in the first 100 orders. Once volume justifies branded packaging and inspection, migrate to self-ship with a clear operational reason. Note this is a Shah Luxury decision per \$9.7 — they may have only one of the two on offer.			

4.2 Hand-off from website to brand partner

When a Stripe webhook confirms payment, the order moves from pending to paid. The hand-off to Shah Luxury happens at this transition.

Drop-ship implementation

```
// /api/webhook handles checkout.session.completed
on('checkout.session.completed', async (session) => {
  const order = await prisma.order.upsert({ ... });

  if (isNewOrder) {
    // 1. Email Shah Luxury fulfillment desk with the order details
```

```

    await sendFulfillmentRequestEmail({
      to: env.SHAH_FULFILLMENT_EMAIL,
      order, items, shippingAddress
    });

    // 2. Append a row to a shared Google Sheet via service-account API
    await appendFulfillmentRow(order);

    // 3. Audit log
    await logAudit({ action: 'order.handoff', resource: order.id });

    // 4. Customer receipt + production-started notice
    await sendOrderConfirmationEmail(user.email, order.id, total);
  }
});

```

Self-ship implementation

- Same trigger as above, but Shah's notification is an internal “Picking ticket” admin page rather than a customer-facing communication.
- Use ShipStation or Shippo to print a label from the order detail.
- Mark as shipped writes `shippedAt`, `carrier`, `trackingNumber`, and transitions status to `shipped`.
- Trigger the shipped email to the customer with tracking link.

4.3 Address validation

Stripe Checkout collects an address but does not verify deliverability. Two layers:

- **Layer 1 — at entry.** Enable Stripe Checkout's “address element with autocomplete” in the Stripe dashboard. Free, uses Google Places suggestions, catches most typos.
- **Layer 2 — post-purchase verification.** In the webhook, call **Smarty** or **Lob** to validate the address against USPS / international postal databases. If the response is `invalid` or `missing_secondary`, transition the order to a new `address_review` state and email the customer to confirm before production starts.

Vendor	Pricing	Coverage
Smarty	\$30/mo for 1k validations, scales linearly	US + international postal
Lob	\$0.05–\$0.10 per call, pay-as-you-go	US + international, also offers print mail
EasyPost	\$0.02 per call, bundled with carrier API	US-only address validation

4.4 Lead-time messaging

Luxury customers tolerate long lead times if they're communicated honestly and consistently. The site advertises “4–6 weeks handcrafted to order.” That must be the truth.

Customer touchpoint	What it says about lead time
Product detail page	"Handcrafted to order · 4–6 weeks · Free insured shipping"
Cart line item	Inline lead-time chip per item
Stripe Checkout receipt	Description includes expected ship date
Order confirmation email	"Your commission has begun" — estimated date the piece begins production
Production started email	"Your piece is now in our craftsmen's hands" — estimated ship date
Shipped email	Tracking number + carrier + actual delivery estimate
Day +3 of expected delivery	Automated check: if not delivered, send a check-in email with concierge contact

4.5 International shipping & customs

Stripe Checkout is currently configured for US, Canada, UK, and Australia. Each has different rules.

Country	Customs / Tax handling
United States	Stripe Tax collects sales tax based on shipping address & [BRAND]'s nexus. No customs.
Canada	GST/HST/PST varies by province; Stripe Tax handles. CAD currency optional via Stripe.
United Kingdom	VAT 20%. For shipments <£135, [BRAND] must register with HMRC and collect VAT at checkout. Above £135, customer pays VAT + duties at delivery (carrier collects).
Australia	GST 10% on imports above AUD \$1,000. [BRAND] must register and remit. Below threshold, no GST.
EU (any country)	IOSS required for orders <€150 — [BRAND] collects VAT at checkout, remits via single filing. Above €150, customer pays at delivery and bounces 8–12% of the time.

Implementation note

Don't enable EU countries in Stripe's address collection until IOSS is registered. Otherwise you'll have a high bounce rate on EU orders and angry customs-fee surprises. Recommended: launch US + Canada only, add UK at month 3, add EU once IOSS is in place.

4.6 Insurance & loss protection

- **Per-shipment declared value** — every parcel with declared value above carrier minimums (usually \$100) gets explicit insurance. FedEx caps free at \$100; UPS caps at \$100; Brink's specializes in high-value diamond shipments.
- **Signature required** — over \$500 declared, always.

- **Two-day delivery floor** — never ground for high-value pieces. Less time in transit means less exposure.
- **Loss process** — written runbook: customer reports non-delivery, [BRAND] files carrier claim, [BRAND] (or insurance) replaces piece with Shah on a rush basis. Define who absorbs the cost.

4.7 Returns & RMA workflow

Custom and made-to-order pieces are usually non-returnable — but customers expect a path to recourse. Define and publish the policy; build the workflow that executes it.

Customer-facing return flow

1. Customer clicks "Request return" on order detail
/profile/orders/[id]
2. Form: reason (dropdown), detail (free text), photo upload
POST /api/account/returns
3. Server creates ReturnRequest with rmaCode "RMA-YYYY-NNNN"
4. Confirmation email to customer with RMA code and return label
(label generated via ShipStation/Shippo)
5. Order status: completed → return_requested
6. Customer ships piece back using the label
7. Carrier scan: status → return_in_transit
8. [BRAND] receives & inspects:
 - If approved → status return_received → refund (Stripe) → refunded
 - If denied → status returned_denied → email customer with explanation
 - If exchange requested → create new Order, mark this exchanged

Admin-side return inspection

- Admin order detail page surfaces the open ReturnRequest with rma code and customer reason.
- Inspection checklist: piece received in original packaging, undamaged, all components present, stone count matches.
- Approve → Stripe refunds the full or partial amount; admin records the refunded amount on the ReturnRequest and audit log captures the decision.
- Deny → email template explains the specific reason; piece is shipped back to customer at customer's expense.

4.8 Warranty repairs & aftercare

The site advertises "Lifetime Warranty" and "Complimentary Lifetime Resizing." Both need a real workflow — neither is currently built.

Customer-initiated warranty request

- /profile/orders/[id] shows a "Request warranty service" link for any order in delivered or completed status.
- Form captures: service type (prong tightening / resizing / replating / stone repair), description, photos.
- Server creates a new entity (RepairRequest, similar shape to ReturnRequest); customer receives a return label and instructions.
- Customer ships piece to [BRAND] (or directly to Shah, depending on §9.6 answer).

- Repair status updates communicate to customer: received → assessed → in progress → completed → shipped.

4.9 Packaging materials & presentation

- **Outer shipping box** — discreet, no jewelry branding visible. Reduces theft in transit.
- **Inner branded box** — the unboxing moment. Magnetic closure box, satin lining, [BRAND]-embossed exterior. Cost: \$4–\$8 per box at MOQ ~500.
- **Polishing cloth** — branded microfiber cloth in box.
- **Certificate of authenticity** — printed card with piece details (material, weight, dimensions, diamond cert if applicable), [BRAND] signature/sigil.
- **Care card** — how to wear, store, and clean.
- **Thank-you note** — handwritten for first 50 orders, printed thereafter.
- **Return packing** — pre-printed return slip with RMA instructions for the rare return case.

Required from Shah Luxury — see Volume 9 §9.6 / §9.7

Confirm whether Shah Luxury offers custom-branded packaging as a service (typical MOQ is 500–1,000 boxes), or whether [BRAND] sources packaging independently. If Shah ships direct, [BRAND]'s branding can only ride along on inserts.

VOLUME 5

Marketing, SEO & Customer Retention

5.1 The marketing stack

Luxury jewelry sells primarily on trust and story, not on discount and urgency. The marketing stack reflects that — heavy investment in editorial content, search visibility, and post-purchase nurture; minimal investment in retargeting, pop-ups, and aggressive paid acquisition. The toolchain is small and chosen for longevity rather than novelty.

Category	Tool	Monthly cost	Owner
Email — transactional	Resend	\$0 (under 3K/mo) → \$20	Engineering
Email — marketing	Klaviyo	\$0 (under 500 contacts) → \$45+	Marketing
Reviews	Junip or Okendo	\$0 (free tier) → \$29+	Marketing
SEO monitoring	Google Search Console	Free	Marketing
Analytics	Vercel Web Analytics + Plausible	\$0 + \$9	Engineering
Schema markup	Custom (JSON-LD in Next.js)	\$0	Engineering
Social	Manual posting + scheduled (Buffer free tier)	\$0	Marketing
Content management	MDX in repo	\$0	Engineering + Marketing

5.2 Email — transactional & lifecycle

Transactional email is the backbone of customer trust. Every order generates 4–7 emails over its lifetime; if any of them fail to arrive, the customer assumes the worst. The stack splits transactional (Resend, fired from server functions) from marketing (Klaviyo, fired from event triggers).

Transactional email matrix (Resend)

Event	Subject line	Trigger	Required from Shah
Order received	Your [BRAND] order — #ORD-1234	checkout.session.completed webhook	—
Production started	Your piece has entered production	Admin marks order in_production	Estimated production days

Event	Subject line	Trigger	Required from Shah
Quality check complete	Your piece has passed quality check	Admin marks order ready_ship	—
Shipped	Your [BRAND] order is on the way — track here	Admin enters tracking number	Carrier & tracking URL format
Out for delivery	Your order arrives today	Webhook from carrier API	Carrier webhook setup
Delivered	Your piece has arrived — care guide inside	Carrier webhook 'delivered' event	—
Password reset	Reset your [BRAND] password	User triggers /api/auth/reset	—
Login from new device	New sign-in to your account	Login detected from new IP/UA	—

Marketing & lifecycle email matrix (Klaviyo)

Flow	Trigger	Sequence length	Goal
Welcome series	Newsletter signup	3 emails over 7 days	Brand story, founder note, soft product intro
Abandoned cart	Cart inactive 1h, 24h, 72h	3 emails	Recover sale without discount; 3rd offers concierge call
Abandoned checkout	Checkout started, not completed	2 emails (1h, 24h)	Address technical or hesitation issues
Post-purchase nurture	Order delivered + 7d / +30d / +90d	3 emails	Care reminder, review request, cross-sell pieces
Re-engagement	No open 90 days	1 email	Editorial content; soft check-in
Birthday	Customer birthday (if collected)	1 email	Personal note (no discount); concierge available
Win-back	No purchase 12 months	2 emails	What's new in collection; concierge call offer

Brand voice rule — no urgency manufacturing

Luxury brands do not use timers, 'only 2 left', flash sales, or fake scarcity. The currency is trust. Klaviyo emails follow this rule: no countdown, no discount codes in subject lines, no all-caps 'LAST CHANCE'. Tone is editorial, restrained, generous.

5.3 SEO foundation

Organic search is the lowest-cost, highest-trust acquisition channel for luxury jewelry — but it takes 6–12 months to build. The site ships with the technical foundation, and Volume 10 includes the editorial calendar for the first year of content.

Technical SEO — what ships with v1

- **Sitemap** — auto-generated at `/sitemap.xml` covering all products, collections, journal posts, and static pages. Regenerated nightly via Vercel cron.
- **robots.txt** — allows all crawlers, points to sitemap.
- **Canonical URLs** — every page declares its canonical via Next.js metadata API.
- **Open Graph + Twitter Card** — every product page emits OG image (1200×630), title, description; every journal post emits OG image with title overlay.
- **Structured data (JSON-LD)** — Product, Offer, BreadcrumbList, Organization, WebSite, Article. Detailed schemas in §5.4.
- **Mobile-first** — Lighthouse mobile score ≥ 90 for all top-level routes.
- **Core Web Vitals** — LCP < 2.5s, INP < 200ms, CLS < 0.1. Tracked via Vercel Web Analytics.
- **Internal linking** — every product page links to its collection, to related pieces, to relevant journal articles.

5.4 JSON-LD structured data

Required schemas, with concrete examples. Add to every page via `<script type="application/ld+json">` tags in the Next.js metadata or `app/layout`.

Product schema (example)

```
{
  "@context": "https://schema.org/",
  "@type": "Product",
  "name": "Sparkle Studs — Silver, Two-tone Diamond, 0.5 ct",
  "image": [
    "https://yourdomain.com/images/sparkle-studs-silver-twt-050-1.jpg",
    "https://yourdomain.com/images/sparkle-studs-silver-twt-050-2.jpg"
  ],
  "description": "Hand-set pavé diamond ball stud earrings...",
  "sku": "sparkle-studs-silver__twt-050",
  "mpn": "SS-SLV-TWT-050",
  "brand": { "@type": "Brand", "name": "[BRAND]" },
  "offers": {
    "@type": "Offer",
    "url": "https://yourdomain.com/customize/sparkle-studs-silver",
    "priceCurrency": "USD",
    "price": "2850.00",
    "priceValidUntil": "2026-12-31",
    "availability": "https://schema.org/InStock",
    "itemCondition": "https://schema.org/NewCondition",
    "shippingDetails": { "@type": "OfferShippingDetails", ... }
  },
  "aggregateRating": {
    "@type": "AggregateRating",
    "ratingValue": "4.9",
    "reviewCount": "47"
  }
}
```

```
}
```

Organization & WebSite schema (site-wide)

```
{
  "@context": "https://schema.org",
  "@type": "Organization",
  "name": "[BRAND]",
  "url": "https://yourdomain.com",
  "logo": "https://yourdomain.com/logo.svg",
  "sameAs": [
    "https://instagram.com/[brand]",
    "https://pinterest.com/[brand]"
  ],
  "contactPoint": {
    "@type": "ContactPoint",
    "contactType": "customer service",
    "email": "concierge@yourdomain.com",
    "areaServed": "Worldwide"
  }
}
```

5.5 Content strategy — the journal

The `/journal` section is the editorial heart of the site. It is what makes Google understand [BRAND] as an authority, not a dropshipper. Three article tracks run in parallel:

Track	Cadence	Examples	Purpose
Craft & materials	Monthly	'How a pavé setting is made', 'Why we choose 14k for everyday wear'	SEO + brand trust
Game & culture	Bi-weekly	'Five pickleball traditions worth keeping', 'Interviews with players'	Audience affinity
Care & styling	Monthly	'How to clean gold jewelry at home', 'Layering pendants'	Customer retention + long-tail SEO

5.6 Reviews & user-generated content

- **Review provider** — Junip or Okendo. Both integrate with Klaviyo and support photo reviews. Junip is cheaper; Okendo has stronger UGC features.
- **Trigger** — Klaviyo review-request email fires 14 days after delivery (long enough that customer has worn the piece).
- **Incentive** — no discount for reviews; instead, photo reviews are featured on product pages with credit and a personal note from the founder.
- **Moderation** — reviews under 3 stars trigger an internal Slack alert and a concierge follow-up before publication; never censored, but always acknowledged.
- **Display** — last 6 reviews on product page, 'See all reviews' opens a modal with full list.
- **Schema** — `aggregateRating` in Product JSON-LD pulls from review average.

5.7 Customer segmentation

Klaviyo segments drive personalized email and inform the concierge team about each customer. Recommended segments to define on day one:

Segment	Definition	Use
VIP — 3+ orders	lifetime_orders ≥ 3 OR lifetime_value > \$10K	First-look at new pieces; concierge proactive outreach
Recent buyer	Order placed within 30 days	Suppress paid retargeting; suppress promo emails
Browser, no buyer	Site visit in last 30d, no order ever	Welcome series + soft introduction emails
At-risk	Bought, no engagement in 90 days	Re-engagement email; soft check-in
Returner	Submitted return in last 90 days	Suppress marketing for 30 days; concierge follow-up
Wishlist active	Has wishlist item, no order on that item	Targeted nurture for that specific piece

5.8 Paid acquisition (year 1: low)

Paid acquisition is intentionally minimal in year one. Luxury brands that over-invest in paid in year one create attribution chaos and dilute organic signal. The recommended allocation:

- **Google Search — branded only** (\$200–500/mo): defend the brand name against competitors bidding on it. Critical from day one.
- **Meta retargeting** (\$300–800/mo): re-engage cart abandoners and journal readers. Pixel installed with explicit consent (Volume 6).
- **Pinterest organic + paid pins** (\$200/mo): jewelry buyers heavily use Pinterest. Visual platform fits the brand.
- **Influencer seeding (gifting)** (\$500/mo budget for pieces): one to two pickleball pros and lifestyle creators per quarter, gifted (not paid) pieces.
- **What to avoid in year one** — TikTok Shop, Amazon, Google Shopping at scale, broad-match search, discount-led paid social.

5.9 Open Graph image generation

Every product page and journal article needs a custom OG image. Next.js 16 supports dynamic OG image generation via the ImageResponse API. Implementation:

```
// app/customize/[slug]/opengraph-image.tsx
import { ImageResponse } from 'next/og';

export const size = { width: 1200, height: 630 };
export const contentType = 'image/png';
export default async function OGImage({ params }) {
```



```
const product = await getProduct(params.slug);
return new ImageResponse(
  (
    <div style={{ display: 'flex', background: '#1A1A1A', width: '100%',
      height: '100%', color: '#fff', padding: 60 }}>
      <img src={product.heroImage} style={{ width: 540, height: 540,
        objectFit: 'cover' }} />
      <div style={{ marginLeft: 60, display: 'flex', flexDirection: 'column' }}>
        <div style={{ fontSize: 28, color: '#C9A96E' }}>[BRAND]</div>
        <div style={{ fontSize: 60, marginTop: 30 }}>{product.name}</div>
        <div style={{ fontSize: 24, marginTop: 20, color: '#A0A0A0' }}>
          ${product.priceUsd.toLocaleString()} · Handcrafted to order
        </div>
      </div>
    </div>
  ),
  { ...size }
);
}
```

VOLUME 6

Legal, Compliance & Trust

6.1 Overview

Selling jewelry online involves three overlapping regulatory regimes: data protection law (GDPR, CCPA, others), consumer goods regulation (FTC Jewelry Guides, customs declarations, hallmarking), and payment processing rules (PCI-DSS, even for hosted checkout). Compliance is not optional and is not merely a checkbox — it is operationalized in every email, every product page, and every page footer.

6.2 Required legal documents

These six documents must exist before launch. They must be reviewed by an attorney licensed in the jurisdiction of incorporation (typically Delaware US or UK Ltd). Off-the-shelf generators (Termly, Iubenda) produce acceptable v1 drafts but should be reviewed.

Document	Purpose	Estimated drafting cost
Terms of Service	Contract between [BRAND] and buyer; limits liability; defines arbitration/jurisdiction	\$500–2,500 attorney; \$39 Termly
Privacy Policy	Required by GDPR Art. 13, CCPA, and most US states; discloses data collection & rights	\$500–2,500 attorney; \$39 Termly
Refund & Returns Policy	Required by FTC and EU Consumer Rights Directive; must state withdrawal rights, conditions, timeframe	\$300–800 attorney
Shipping Policy	Customs, lead times, insurance, lost package process — separate from returns	\$200–500 attorney
Cookie Policy	Lists every tracking cookie/pixel with purpose; links to consent management	Included in Privacy Policy
Acceptable Use Policy	Protects [BRAND] from abusive accounts & bots	\$200–500 attorney

Required from owner — see Volume 9 §9.10

Entity selection (LLC, Inc., Ltd.) and jurisdiction of incorporation must be decided before Terms of Service can be drafted. Without this, customers have no clear contractual counterparty.

6.3 GDPR (European customers)

GDPR applies the moment [BRAND] takes a single order from an EU customer or runs ads targeting them. Non-compliance fines reach 4% of global revenue. The following must be live before any EU-targeted advertising:

- **Lawful basis for processing** — contract performance for orders, consent for marketing, legitimate interest for fraud prevention. Document each in the Privacy Policy.
- **Consent management platform** — cookie banner with granular toggles (necessary / analytics / marketing). Recommended: Cookiebot, iubenda, or open-source Klaro. Cost: \$14–\$50/mo.
- **Data subject access request (DSAR) endpoint** — at `/api/account/dsar`, must produce a downloadable export of all customer data within 30 days of request.
- **Right to erasure** — at `/api/account/delete`, must hard-delete customer data within 30 days (except records required by tax law: orders, invoices for 7 years).
- **Data Processing Agreement (DPA)** — signed with every processor: Stripe, Resend, Klaviyo, Vercel, Turso, Sentry. All major providers offer a standard DPA via their dashboards.
- **Standard Contractual Clauses (SCCs)** — required for any processor outside the EU/EEA (most US providers).
- **Data Protection Officer** — not required for low-volume e-commerce; review at >5,000 EU customers.
- **Breach notification** — within 72 hours to supervisory authority. See incident-response runbook in Volume 7.

6.4 CCPA (California customers)

- **'Do Not Sell or Share My Personal Information' link** — must be in the footer for California IPs. Functionally similar to GDPR opt-out.
- **Privacy Policy disclosure** — categories of personal information collected, sources, purposes, third parties.
- **Consumer rights** — right to know, right to delete, right to opt out, right to non-discrimination.
- **Global Privacy Control (GPC) signal** — must honor automatically; treat as a valid opt-out.

6.5 FTC Jewelry Guides (US — every claim matters)

The FTC's Jewelry, Precious Metals, and Pewter Industries Guides specify how every jewelry product may be described. Violations can trigger enforcement action and class-action lawsuits.

- **Gold purity** — '14k gold' must be at least 14 karats. Plated items must say 'gold-plated', 'gold-filled', or 'gold-tone'. Never 'gold' alone for plated items.
- **Diamond claims** — total carat weight (TCW) must be accurate within 0.005 ct. Color and clarity grades must use GIA, AGS, or IGI scales — never made up.
- **Lab-grown diamonds** — must be disclosed as 'laboratory-grown', 'laboratory-created', or '[manufacturer name]-created' on the product page AND in marketing copy. Cannot be sold as 'real diamonds' without qualifier.
- **'Hand-crafted' / 'hand-made'** — only permitted if substantially produced by hand. Cast pieces with machine setting cannot use these terms.
- **'Custom' / 'bespoke'** — must involve customer-specific design decisions. Configurator combinations are 'personalized', not 'custom'.
- **Origin claims** — 'Made in USA' requires substantially all manufacturing in the US. If Shah manufactures in India, origin must be disclosed.

- **Warranty disclosure** — must state what is and isn't covered, duration, claim process. See Volume 4 §4.8.

Required from Shah Luxury — see Volume 9 §9.4

Authoritative metal purity, diamond grading methodology, and country of manufacture for every piece in the catalog. Without this, every product page is at risk of an FTC violation. Diamond claims must be backed by certification documents.

6.6 Sales tax & VAT

Tax handling is the most operationally complex part of e-commerce. Recommended approach: outsource it. Stripe Tax handles sales tax in 50 US states + international VAT automatically for an additional 0.5% of the transaction. The alternative — TaxJar or Avalara plus an accountant — is more expensive and more error-prone.

- **US sales tax** — nexus is established by physical presence or by economic activity (\$100K+ in sales or 200+ transactions per state, varies). At launch, only the home state nexus applies; review quarterly.
- **EU VAT** — IOSS (Import One-Stop Shop) registration if shipping to EU; otherwise customer pays VAT on import (worse experience). Cost: ~\$200 setup + accountant fees.
- **UK VAT** — required at £85K UK sales/year; recommended to register voluntarily for VAT reclaim.
- **Duties** — Stripe does not handle import duties. Recommended: Zonos or Easyship plug-in shows duties at checkout (DDP — delivered duty paid) so customer is not surprised. Critical for luxury price points.

6.7 PCI-DSS scope

- **Stripe-hosted Checkout** places [BRAND] in SAQ-A scope — the lowest PCI obligation. Customer card details never touch the [BRAND] server.
- **SAQ-A requirements** — TLS on all pages, secure development practices, no card data in logs, annual self-assessment.
- **Do not** implement Stripe Elements with on-page card fields — that raises PCI scope to SAQ-A-EP.
- **Do not** log Stripe webhook payloads in full — they may contain customer email and card last4. Mask in Sentry.

6.8 Accessibility — WCAG 2.1 AA

ADA Title III lawsuits against e-commerce sites have surged. Compliance is both ethically right and legally protective. Target: WCAG 2.1 Level AA.

- **Color contrast** — text 4.5:1 against background; large text 3:1. Auditable in browser dev tools.
- **Keyboard navigation** — every interactive element reachable and operable with keyboard alone.
- **Screen reader labels** — every image has descriptive alt text (not 'image123.jpg'); every form field has a visible or aria-label.
- **Focus indicators** — every interactive element shows a clear focus ring when focused.

- **Motion preferences** — respect prefers-reduced-motion. GSAP/Framer animations skip when set. Already implemented in current codebase.
- **Skip links** — 'Skip to main content' link at top of every page.
- **Heading hierarchy** — one h1 per page; no skipped levels.
- **Audit** — automated (axe-core in CI) catches ~30%; manual audit by an accessibility consultant annually catches the rest. Budget: \$1,500–\$3,500.

6.9 Trust signals (on-page)

- **Certifications visible on product pages** — GIA diamond cert reference, metal hallmark, conflict-free sourcing statement.
- **Concrete shipping & return policy** — '30-day returns. No restocking fee.' near every Add to Cart button.
- **Security badges** — Stripe-secured checkout badge, SSL indicator, BBB (if applicable).
- **Real photos with credit** — founder photo, atelier/workshop photos, customer photos with permission.
- **Contactable** — concierge email and phone number visible in footer; response-time commitment.
- **Reviews** — visible aggregate rating + last 6 reviews on product pages. See §5.6.

VOLUME 7

Security, Authentication & Audit

7.1 Threat model

A luxury jewelry e-commerce site faces a specific attack surface. The realistic threats — ranked by likelihood times impact:

Threat	Likelihood	Impact	Primary mitigation
Credential stuffing on customer accounts	High	Medium	Argon2id + account lockout + 2FA option
Stripe webhook spoofing	Medium	High (free orders)	Webhook signature verification
Admin account compromise	Low	Critical	Required 2FA + IP allow list + audit log
XSS in product reviews	Medium	Medium	Server-side sanitization (DOMPurify)
Card testing fraud	High	Low (Stripe absorbs)	Stripe Radar enabled
Price-tampering on cart	Medium	High	Server-side price recompute (already implemented)
Coupon abuse (multi-use)	Medium	Low	Per-user redemption tracking; rate-limit
DoS — scraping for pricing	High	Low	Rate limiting + WAF + Cloudflare in front of Vercel
Supply-chain attack via npm	Low	Critical	Dependabot + lockfile audits + Socket.dev
Insider threat (leaked secrets)	Low	Critical	Vault, no .env in repo, secret rotation

7.2 Authentication architecture

The current implementation uses custom JWT auth — chosen for control and zero external dependency. The architecture:

- **Passwords** hashed with Argon2id (memory 64 MB, time 3, parallelism 4). Stronger than bcrypt for the same compute budget.
- **Access tokens** are signed JWTs, 15-minute expiry, stored in HTTP-only Secure SameSite=Strict cookies.

- **Refresh tokens** are random 64-byte tokens (not JWTs), SHA-256 hashed at rest, 7-day expiry, family rotation with reuse detection.
- **Sessions** for admin users are tracked in a separate Session table with userAgent + IP; allow explicit 'sign out of all devices'.
- **Account logout** — 5 failed logins triggers 15-minute logout; logoutCount tracked.
- **Password reset** — single-use random token, SHA-256 hashed at rest, 1-hour expiry.

Should we replace it with Clerk or NextAuth?

Probably not for v1. The custom implementation is correct, audited, and runs on no external dependency. Migrating to Clerk later is straightforward (1–2 days of work) if 2FA enrollment, social login, or passkey support become important. The trigger to migrate would be: more than ten support tickets per month related to login problems, or a need for SAML/SSO for B2B sales.

7.3 Authorization model

- **Three roles** — customer, admin, super_admin. Stored on the User table.
- **Permission checks** — every admin route wraps in `requireRole('admin')`; every customer route wraps in `requireAuth()`. Both helpers throw 401/403 before any business logic.
- **Object-level checks** — when reading an Order or Address, verify `userId === currentUser.id` before returning. No exceptions.
- **Admin 2FA enrollment** — required at first login for any user with admin or super_admin role. TOTP via otpauth library; backup codes generated and shown once.
- **Admin IP allow list** — optional environment variable `ADMIN_IP_ALLOWLIST`. Useful once team is established and offices are stable.

7.4 Secrets management

Secrets policy: nothing in the repo, nothing in plain logs, nothing without a rotation plan. The current implementation lazily resolves secrets at request time (not at module load), which means a missing secret fails the request loudly rather than crashing the build.

- **Production secrets** — Vercel encrypted environment variables. Rotated quarterly for high-value secrets (`JWT_ACCESS_SECRET`, `FIELD_ENCRYPTION_KEY`, `STRIPE_SECRET_KEY`).
- **Local development** — `.env.local` in `.gitignore`. Provided template `.env.example` with placeholder values only.
- **CI/CD** — GitHub Actions reads secrets from repository secrets, never echoes them in logs.
- **Audit trail** — every secret rotation is logged with timestamp + who rotated + reason in a `SECRETS_LOG.md` file (private, not committed).
- **Webhook secrets** — `STRIPE_WEBHOOK_SECRET`, `RESEND_WEBHOOK_SECRET`. Verified on every webhook request; mismatches trigger Sentry alert.
- **Field encryption key** — 32-byte hex string, used for `encryptField` / `decryptField` on `AddressLine1` and `PhoneNumber`. If rotated, requires re-encryption of all rows.

7.5 Audit log

The AuditLog table records every privileged or sensitive action. Retained for 2 years for incident response and compliance. Cannot be deleted by application code — only via direct DB access.

```
// Recommended schema additions (also in Appendix A)
model AuditLog {
  id          String    @id @default(cuid())
  actorId     String?   // null for system actions
  action      String    // 'order.cancel', 'admin.user.role-change', 'auth.password-reset'
  resourceType String?   // 'Order', 'User', 'Coupon'
  resourceId  String?
  metadata    Json?     // additional context
  ipAddress   String?
  userAgent   String?
  createdAt   DateTime  @default(now())

  @@index([actorId, createdAt])
  @@index([action, createdAt])
  @@index([resourceType, resourceId])
}

// Events to log:
// - auth.login.success / auth.login.fail / auth.password-reset / auth.2fa-enroll
// - admin.user.role-change / admin.user.lockout-clear
// - admin.order.refund / admin.order.cancel / admin.order.status-change
// - admin.coupon.create / admin.coupon.deactivate
// - data.export (DSAR) / data.delete (right-to-erasure)
// - settings.secret-rotated
```

7.6 Rate limiting

- **Edge layer** — Vercel WAF Pro tier or Cloudflare in front. Blocks volumetric attacks and known bad IPs before they hit the application.
- **Application layer — Upstash Redis.** In-process limiters do not work in a serverless multi-instance environment.
- **Per-endpoint limits:**

Endpoint	Limit	Window	On exceed
/api/auth/login	5	5 min	429, log to AuditLog
/api/auth/register	3	1 hour per IP	429
/api/auth/reset	3	1 hour per email	Silent 200 (avoid enumeration)
/api/checkout	10	1 hour per IP	429
/api/account/dsar	1	24 hours per user	429
/api/contact	5	1 hour per IP	429
/api/* (general)	100	1 min per IP	429

7.7 Content Security Policy (CSP)

CSP header should be applied via middleware. Required entries (note: Stripe + GA4 + Klaviyo + Sentry all need explicit allowances):

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-{nonce}'
    https://js.stripe.com
    https://www.googletagmanager.com
    https://static.klaviyo.com;
  style-src 'self' 'unsafe-inline'
    https://fonts.googleapis.com;
  font-src 'self' https://fonts.gstatic.com;
  img-src 'self' data: blob:
    https://*.stripe.com
    https://yourdomain.com;
  connect-src 'self'
    https://api.stripe.com
    https://*.ingest.sentry.io
    https://a.klaviyo.com;
  frame-src https://js.stripe.com https://hooks.stripe.com;
  object-src 'none';
  base-uri 'self';
  form-action 'self' https://checkout.stripe.com;
  frame-ancestors 'none';
  upgrade-insecure-requests;
```

7.8 Incident response runbook

When something goes wrong — a breach, an outage, a fraud spike — the steps and timing matter for both customer trust and regulatory compliance. The runbook lives in `/docs/INCIDENT_RESPONSE.md` and is followed without modification under pressure.

Severity	Examples	First action (within)	Communication
P0 — Critical breach	PII leak, account takeover, Stripe key leak	15 minutes	Notify counsel, prepare 72h GDPR notification
P1 — Site outage	Site down, checkout broken, login broken	30 minutes	Status page update; tweet if >30 min
P2 — Degraded	Slow checkout, partial email delivery	2 hours	Internal Slack; status page if customers affected
P3 — Anomaly	Fraud spike, scraping spike	Next business day	Internal review

Standard incident actions (any severity)

- Open an incident channel (`#inc-yyyymmdd`) in Slack.
- Assign one Incident Commander — the single decision-maker for the duration.
- Snapshot logs, screenshots, dashboards — preserve evidence before remediating.
- Determine scope: how many customers? what data? when did it start?

- Stop the bleeding before forensics — rotate the compromised secret, take the bad endpoint down.
- Customer communication template ready in `/docs/templates/breach-notification.md`.
- Post-incident review within 5 business days; runbook updated with learnings.

VOLUME 8

Performance, Caching & Scale

8.1 Performance budgets

Every page has explicit performance budgets enforced via Lighthouse CI in the build pipeline. A regression beyond budget fails the deploy.

Metric	Budget	Why
Largest Contentful Paint (LCP)	≤ 2.5s	Google ranking + perceived load
Interaction to Next Paint (INP)	≤ 200ms	Replaces FID in CWV 2024
Cumulative Layout Shift (CLS)	≤ 0.1	Editorial layouts must not jump
Total page weight (homepage)	≤ 1.5 MB	Including images
Total page weight (product page)	≤ 2.0 MB	Including hero video
JS bundle (gzipped)	≤ 180 KB initial	Code-split routes
Time to First Byte (TTFB)	≤ 600ms	Server response budget
Lighthouse Performance score	≥ 90 mobile	CI gate

8.2 Image strategy

Images are the single largest performance lever for a jewelry site. The strategy uses Next.js `next/image` for automatic optimization, with these conventions:

- **Master assets** stored as high-resolution JPGs (or AVIF where supported by source). PNG only for assets requiring transparency.
- **Catalog imagery** served via `/_next/image` at 4 widths: 320, 640, 1080, 1920. AVIF served to supporting browsers; WebP fallback; JPG last resort.
- **Hero images** use `priority` for above-fold; `loading="lazy"` below.
- **Configurator imagery** preloaded for common variants on hover; transitions use `opacity-only` (no layout shift).
- **Image domains** — only `yourdomain.com`, no hot-linking from external sources. Configured in `next.config.ts`.
- **OG images** generated dynamically via `ImageResponse` (Volume 5 §5.9), cached for 7 days.

8.3 Video strategy

- **Master format** — H.264 MP4 with AAC audio. 1080p maximum; mobile gets 720p variant.

- **Encoding** — two-pass VBR, CRF 22–24 for quality. Target bitrate 4–6 Mbps for 1080p.
- **Hosting** — Mux or Cloudflare Stream for HLS adaptive bitrate. ~\$1/1000 minutes streamed.
- **Self-hosting** — acceptable for under 10 GB/mo total transfer; Vercel Pro bandwidth covers it. Beyond that, costs spiral.
- **Scrub videos in configurator** — preloaded with `preload="metadata"`, played frame-by-frame on slider drag. Use `video.currentTime` rather than playback.
- **Hero video** — muted autoplay, loop, playsinline. Poster image for first frame.

8.4 Caching layers

Layer	What's cached	TTL	Invalidation
Vercel Edge CDN	Static HTML for product pages, /collections, /journal	1 year (Vercel)	On deploy + tag-based revalidation
Next.js Data Cache	Fetched products, collections, journal entries	1 hour (default)	<code>revalidateTag('products')</code>
Browser HTTP cache	JS, CSS, fonts, images	1 year (immutable hashes)	Hash changes on rebuild
Stripe Checkout session	Server memory	30 minutes	Auto-expires
User session	Database (Session row)	8 hours	On logout or expiry
Rate limit counters	Upstash Redis	Window-defined	Auto-expires

Cache invalidation triggers

```
// app/api/admin/products/[id]/route.ts – when admin edits a product
import { revalidateTag, revalidatePath } from 'next/cache';

await prisma.product.update({ where: { id }, data });
revalidateTag('products');
revalidateTag(`product:${id}`);
revalidatePath(`/customize/${slug}`);
revalidatePath('/collections');
revalidatePath('/sitemap.xml');
```

8.5 Database scaling plan

Turso (libSQL) handles the projected first-year traffic on the Hobby tier (\$0). The pricing curve and migration plan:

Tier	Cost	Reads/ mo	Writes/ mo	Storage	When to upgrade
Turso Free	\$0	1B	25M	9 GB	Day 1

Tier	Cost	Reads/ mo	Writes/mo	Storage	When to upgrade
Turso Scaler	\$29	1B	100M	24 GB	>25M writes (orders + analytics)
Turso Pro	\$99	Unlimited	Unlimited	100 GB	>100K monthly visitors
Self-host Postgres	~\$50	Unlimited	Unlimited	Per disk	Compliance requires region-pin

8.6 Observability stack

Concern	Tool	Cost	What it answers
Errors	Sentry	\$0 → \$26	What's breaking? For whom? Where in code?
Performance	Vercel Web Analytics	\$0 → \$10	Real-user CWV; page-by-page LCP/INP/CLS
Traffic	Plausible (privacy-first)	\$9	How many visitors? Where from? What pages?
Uptime	Better Stack / Cronitor	\$0 → \$24	Is the site up? Last 5 min?
Logs	Vercel Logs + Logflare	\$0 → \$13	What did the server do? What did the webhook receive?
DB query perf	Turso console	\$0	Slow queries; missing indexes

8.7 Scaling assumptions

These numbers anchor capacity decisions. Revisit quarterly:

- **Year 1 baseline** — 5,000 monthly visitors, 50 orders/mo, \$50K AOV.
- **Year 1 stretch** — 25,000 monthly visitors, 250 orders/mo, \$50K AOV. Still well within Vercel Pro and Turso Free tiers.
- **Year 2** — 100K visitors, 1K orders/mo. Migration triggers: Vercel Pro → Enterprise (if outgrowing 1 TB bandwidth), Turso Free → Scaler.
- **Black Friday spike** — assume 10× normal traffic for 72 hours. Vercel handles this transparently; rate limiters must scale (Upstash Redis is fine).

8.8 When to add a search engine

Initially the catalog is small (<50 SKUs) and database LIKE queries are sufficient. Add Algolia or Typesense when:

- Catalog exceeds ~200 SKUs.

- Users report 'I can't find X'.
- Multi-attribute filtering (metal, stone, price range, style) becomes important — slower at DB layer.
- Need for typo tolerance and synonym handling.

Algolia is the obvious choice — 99% of luxury sites use it. Cost: free up to 10K records / 10K searches monthly; \$1/1K searches above. Typesense is open-source and cheaper if self-hosted.

VOLUME 9

Brand Partner Specification & Requests

9.0 How to use this volume

Volume 9 is structured so it can be sent to Shah Luxury as a single self-contained document. Every section contains numbered questions; every question expects a specific answer or document. The intention is to gather, in one round trip, everything required to operate the site at production quality.

Recommended workflow: copy this volume into a shared Google Doc, share with Shah's operations and design lead, give a two-week turnaround. Schedule a one-hour follow-up to resolve open items.

About this list

Every question marked with a Q. icon is an actionable request to Shah Luxury. Sub-questions help Shah scope each answer. Where a sample answer is expected, [BRAND] has noted the data format and units.

9.1 Pricing — the urgent block

Until Shah confirms wholesale prices and the MSRP policy, [BRAND] cannot accept live orders. The checkout API recomputes every line item server-side from the values in the code, so whatever sits there is what the customer pays. Today those values are placeholders.

- Q. Wholesale price per piece** — for every SKU and every variant in the configurator (metal × stone × treatment × size), what is the wholesale invoice price [BRAND] will pay Shah Luxury? Format: USD; effective date; valid-through date.
- Q. Pricing model** — is wholesale fixed per SKU, or commodity-indexed (metal weight × spot price + labor + stone cost + margin)? If indexed, please share the formula.
- Q. MSRP policy** — does Shah Luxury require [BRAND] to maintain a Minimum Advertised Price? If yes, what is the MSRP per SKU?
- Q. Discount permission** — can [BRAND] run promotional discounts? If yes, what is the maximum permitted discount, and are there blackout windows or category restrictions?
- Q. Diamond pricing** — diamond surcharges are currently linear (\$1,000 per 0.5 ct). Is this consistent with Shah's actual cost structure? For each diamond size offered (0.5, 1.0, 1.5, 2.0 ct), what is the wholesale surcharge?
- Q. Metal price reset cadence** — gold spot price varies daily. How often does Shah update wholesale prices? Will Shah honor a quoted price for N days?
- Q. Currency** — wholesale priced in USD only, or also in INR/EUR? Are there FX risk-sharing terms?
- Q. Volume tiers** — is there a volume-discount tier (e.g., 10 pieces/month gets 5% off)? At what thresholds?

9.2 Catalog & product information

- Q. Complete SKU list** — every piece Shah Luxury is willing to manufacture for [BRAND]. Include: SKU code, name, silhouette (ball pendant, ball earring, paddle, court), wear type, base metal options.
- Q. Per-SKU material spec** — for each SKU, list permitted metals (10k yellow, 14k yellow, 14k rose, 14k white, 18k yellow, sterling silver, etc.), with which combinations are stock vs. made-to-order.
- Q. Per-SKU stone spec** — diamond cut, color grade (G–H typical), clarity grade (VS1–SI1 typical), carat weights offered. Lab-grown or natural?
- Q. Dimension & weight** — for each variant, exact dimensions (mm) and metal weight (g). Required for product page detail and shipping calculations.
- Q. Hallmarking** — what hallmark stamps appear on each piece? Country of hallmarking (US, UK, India)?
- Q. Country of manufacture** — required by FTC and customs. Is it consistent across SKUs?
- Q. Production lead time** — for each SKU, business days from order receipt to ready-to-ship. Differentiate stock items vs. made-to-order.
- Q. Max concurrent production capacity** — how many pieces can Shah produce per week? Per month? Is there a queue we should anticipate at peak?
- Q. Product photography** — does Shah supply white-background catalog photography? Lifestyle photos? What resolution and licensing terms?
- Q. 3D model / spec drawings** — for configurator preview, does Shah have CAD files or 3D models that [BRAND] can use? If not, what is the cost to commission?

9.3 Manufacturing & customization capabilities

- Q. Engraving** — what surfaces can be engraved on each piece? Maximum characters? Fonts available? Is engraving included or surcharged? Lead-time impact?
- Q. Birthstone substitution** — can a diamond be substituted with a birthstone of equivalent size? Which birthstones are supported?
- Q. Custom carat sizes** — beyond the 0.5/1.0/1.5/2.0 ct grid, can Shah accommodate custom carat weights? Lead time? Surcharge?
- Q. Custom metal alloys** — beyond standard 10/14/18k yellow/rose/white, are exotic alloys (palladium, platinum) available? At what surcharge?
- Q. Sizing — earrings, rings, bracelets** — what sizes are offered? Is resizing a service Shah provides?
- Q. Plating & finishes** — beyond plain/pavé, what surface treatments are available (matte, brushed, hammered, rhodium-plated)?
- Q. Special-occasion turnaround** — for an urgent gift (anniversary, proposal), can Shah expedite a specific SKU to 5 business days? Surcharge?
- Q. Re-make policy** — if a piece arrives defective or wrong, what is Shah's remake cadence? Who pays?

9.4 Certifications, sourcing & compliance

- Q. Diamond certification** — which grading body (GIA, IGI, AGS) certifies stones ≥ 0.50 ct? Is certification included? At what carat threshold are stones certified vs. uncertified?
- Q. Metal purity assay** — for each metal claim ('14k gold'), how does Shah verify purity? X-ray fluorescence? Acid test? Independent assay?
- Q. Conflict-free declaration** — can Shah supply a signed Kimberley Process / conflict-free declaration for diamonds? RJC (Responsible Jewellery Council) membership?
- Q. Recycled metals** — what percentage of metal is recycled? Is third-party verification available (SCS Global, Eco-Age)?
- Q. Lab-grown disclosure** — for any lab-grown diamonds, confirm grower name, growth method (HPHT, CVD), and IGI certification.
- Q. Country of origin** — diamonds origin, metal origin. Required for some EU markets.
- Q. REACH / RoHS compliance** — relevant for any metals or finishes if shipping to EU.
- Q. Insurance during manufacture** — while a piece is in Shah's workshop pending shipment to a customer, who insures the goods? At what valuation?

9.5 Quality assurance & inspection

- Q. QC process** — what is Shah's pre-ship inspection routine? Visual? Magnification? Gemological?
- Q. QC documentation** — does Shah provide a QC checklist or photo with each ship? Can [BRAND] have visibility into the QC step?
- Q. Defect rate** — historical defect rate per 1000 pieces? Most common defects?
- Q. Independent inspection option** — would Shah allow a third-party gemologist to inspect a sample of shipments quarterly?
- Q. Quarantine policy** — if [BRAND] identifies a recurring defect, what is Shah's investigation and corrective-action turnaround?

9.6 Shipping & fulfillment model

- Q. Drop-ship or self-ship?** — does Shah ship the piece directly to the [BRAND] customer, or to [BRAND] who then re-ships?
- Q. If drop-ship:** can the shipping label show '[BRAND]' as sender (white-label), or must it show Shah Luxury?
- Q. Carriers used** — for domestic US and international shipments, which carriers does Shah use? Are there preferred carriers for jewelry?
- Q. Insurance during transit** — what carrier insurance is in place? At what valuation? Who is the claim-able party if a package is lost — Shah or [BRAND]?
- Q. Tracking API access** — can Shah provide tracking numbers via API, or only manually? Webhook on delivery events?

- Q. **International shipping** — which countries does Shah ship to? Duties & taxes — DDP or DDU?
- Q. **Lost / damaged in transit** — what is Shah's policy? Who pays replacement? How long is claim window?
- Q. **Restricted destinations** — are there countries where Shah will not ship (sanctions, regulatory)?
- Q. **Signature on delivery** — required for all shipments? For shipments over a value threshold?

9.7 Packaging, branding & unboxing

- Q. **Packaging supplied** — what packaging does Shah include with each piece? Inner box, outer mailer, polishing cloth?
- Q. **Custom-branded packaging** — does Shah offer custom-branded boxes (with [BRAND] logo & design)? MOQ? Cost per unit? Lead time for first order?
- Q. **Certificate of authenticity** — is a certificate included? Can it be co-branded with [BRAND]?
- Q. **Inserts / collateral** — can [BRAND] ship inserts (care card, thank-you note) to Shah for inclusion?
- Q. **Box-in-box presentation** — for gift orders, can Shah include gift wrap or a separate outer gift box?
- Q. **Sample request** — can [BRAND] receive sample packaging for photography and approval?

9.8 Returns, warranty & ongoing service

- Q. **Return acceptance** — does Shah accept returns from customers? At what condition? Restocking fee?
- Q. **Return shipping address** — for returns, where does the customer ship to — Shah directly, or [BRAND]'s address?
- Q. **Refurbishment** — for a returned worn piece, can Shah refurbish for resale? At what cost?
- Q. **Warranty terms** — what does Shah's warranty cover (manufacturing defects, stone loosening, plating wear)? Duration? Free or surcharged?
- Q. **Out-of-warranty repair** — what services and at what cost (resize, polish, restring, prong tighten)?
- Q. **Customer-direct repair** — does Shah accept repair items shipped directly by customers, or must all repairs flow through [BRAND]?
- Q. **Repair turnaround** — typical days from receipt to ship-back?

9.9 Commercial terms & legal framework

- Q. **Payment terms** — net-7? net-30? credit limit? deposit required for first order?
- Q. **Minimum order quantity** — per order, per month? Penalty for shortfall?
- Q. **Exclusivity** — does [BRAND] have exclusive rights to sell pickleball-themed jewelry, or can Shah supply competitors? Within which geographic markets?
- Q. **Brand usage** — can [BRAND] reference 'Crafted by Shah Luxury' or similar attribution? Required, permitted, or prohibited?

- Q. Image & design IP** — who owns the design IP for each piece? Can [BRAND] register design patents on configurator combinations?
- Q. Confidentiality** — is a mutual NDA in place? If not, can one be signed?
- Q. Liability** — if a customer is injured by a piece (allergy, sharp edge), how is liability apportioned? Recommended: a Manufacturing & Supply Agreement covering this.
- Q. Insurance coverage** — does Shah maintain product liability insurance? Coverage amount? Can [BRAND] be named additional insured?
- Q. Termination** — notice period to wind down the relationship? Inventory clearance terms?
- Q. Dispute resolution** — arbitration venue? Governing law?

9.10 Day-to-day operations

- Q. Order placement channel** — does [BRAND] place orders by API? Email? Portal? What's the SLA for order acknowledgment?
- Q. Order data format** — what fields does Shah need per order (SKU, variant spec, customer address, engraving, gift message)?
- Q. Single point of contact** — who at Shah is the operational POC for [BRAND]? Backup contact?
- Q. Time zone & response SLA** — Shah's business hours, response-time commitment for ops queries?
- Q. Escalation path** — for urgent issues (wrong shipment, missed deadline), who is escalation contact? Phone number?
- Q. Inventory visibility** — can [BRAND] see real-time stock for stocked SKUs? Or is everything made-to-order?
- Q. Production calendar** — Shah's holiday calendar and shutdown periods (Diwali, Christmas) — important for customer expectations.
- Q. Capacity announcement** — for [BRAND] launches and PR campaigns, how much advance notice does Shah need to ramp capacity?

9.11 Owner-side decisions required (not Shah)

These items are not Shah's to provide — they require [BRAND] owner decisions. Listed here for completeness because they block downstream operational setup.

- Q. Legal entity** — LLC / Inc. / Ltd. — jurisdiction of incorporation? (Required before Stripe account, terms of service, payment terms.)
- Q. Tax ID** — EIN, VAT number, GST number as applicable.
- Q. Bank account** — for Stripe payouts. Currency? Bank?
- Q. Trademark** — has the [BRAND] name been searched (USPTO, EUIPO, IPI)? Application filed?
- Q. Domain** — confirmed and registered? Registrar? With privacy protection?
- Q. Business address** — for shipping returns and legal notices.

- Q. Phone number** — VoIP number for customer concierge (Google Voice, OpenPhone)?
- Q. Pricing positioning** — what is the margin target [BRAND] requires on each piece (typical luxury DTC: 4–5× wholesale)?
- Q. Launch market** — US only at launch, or US + UK? US + worldwide? Drives shipping / tax / compliance scope.

VOLUME 10

Business Operations & Launch Plan

10.1 Cost summary

All costs in USD, current to 2026. Assumes US-based incorporation, US-primary market with limited international shipping. Excludes Shah Luxury wholesale cost (paid per piece, treated as COGS).

One-time launch costs

Item	Cost (USD)	Notes
Domain name (.com)	12–20/yr	Annual; included in recurring
Legal entity formation	150–800	DIY (LegalZoom) to attorney-led
Trademark search + filing	275–2,000	USPTO filing fee + optional attorney
Terms / Privacy / Returns drafting	1,500–5,000	Termly + attorney review
Manufacturing & Supply Agreement w/ Shah	1,500–4,000	Attorney drafting
Photography (lifestyle & lookbook)	3,000–10,000	If not supplied by Shah
Video (hero + scrub videos)	2,000–6,000	Production + editing
Brand identity / logo design	500–5,000	If not already complete
Accessibility audit	1,500–3,500	Pre-launch
Initial packaging order (500 boxes)	2,000–4,000	Branded boxes + cards
Initial inventory (if not pure MTO)	10,000–50,000	Cash-tied; depends on model
One-time total (low)	\$22,475	Lean launch
One-time total (high)	\$90,320	Full kit-up

Recurring costs — monthly

All entries below assume paid, professional tiers. Free tiers are intentionally avoided — a brand partnered with Shah Luxury cannot run on the same plans a side project uses. Paid tiers buy SLAs, priority support, signed DPAs, and the headroom to absorb a launch spike without surprises.

Service	Tier	Monthly (USD)	Notes
Vercel Pro	Hosting + edge + analytics	40–80	2 seats; bandwidth + functions headroom
Cloudflare Pro	WAF + DDoS in front of Vercel	20	Bot management; required at this brand tier

Service	Tier	Monthly (USD)	Notes
Turso Scaler	Production database — paid	29	100M writes, 24 GB, point-in-time recovery
Upstash Redis Pay-as-you-go	Rate limit + cache	10–30	Production tier; not free
Stripe	Payment processing	0	Per-transaction only (see below)
Stripe Tax	Tax automation	0	0.5% per transaction
Stripe Radar for Fraud Teams	Enhanced fraud screening	0	+\$0.05 per screened transaction
Sentry Team	Error monitoring	26	Production-grade alerting
Resend Pro	Transactional email	20–90	Includes dedicated IP option at scale
Klaviyo Email	Marketing email — paid tier	45–150	Tier rises with contact list size
Klaviyo SMS (recommended)	Order updates + abandoned cart	0–60	Per-message + monthly minimum
Plausible Business	Privacy-first analytics	19	Multi-site + custom events
Junip Pro / Okendo Essentials	Reviews — paid	29–99	Photo reviews + Klaviyo flow integration
Better Stack Pro	Uptime + status page + log mgmt	24–50	Public status page included
Cookiebot / Iubenda Pro	Consent management	14–50	Required for EU; paid tier for full audit log
1Password Business	Team password manager	24	3 seats × \$8; required for shared secrets
GitHub Team	Code hosting + Actions + SSO	12	3 seats × \$4; required for protected branches
Dependabot / Socket.dev	Supply-chain scanning	10	Free for solo; paid at team tier
Vercel Web Analytics Pro	Real-user CWV	10	Included with Pro; +\$10 for retention
Domain renewal	Amortized	1–2	Annual paid yearly
Accountant / bookkeeper	Monthly close + tax prep	250–600	Outsourced; firm with e-commerce experience
Monthly total (low)		\$533	Day-1 paid baseline

Service	Tier	Monthly (USD)	Notes
Monthly total (high)		\$1,365	Mature operation, 25K visitors/mo

Per-transaction costs

Cost	Rate	On \$1,500 order
Stripe (US card)	2.9% + \$0.30	\$43.80
Stripe (international card)	+1.5%	+\$22.50
Stripe Tax	0.5% on tax collected	Variable
Shipping (insured)	\$15–\$50	Carrier-dependent
Packaging materials	\$8–\$15	Box + cloth + cards
Klaviyo email cost	<\$0.01	Negligible per order

Margin reality check

At a \$1,500 selling price with Shah wholesale of \$375 (typical 4× markup) and ~\$80 in per-order operating costs (Stripe + shipping + packaging), gross margin per order is roughly \$1,045 (~70%). This is healthy for jewelry DTC. The lever to protect: avoid discounts. Each 10% off the price removes \$150 from the margin — equivalent to 14% of the gross profit.

10.2 90-day launch roadmap

Days 1–30 — Foundation & Brand Partner Sign-off

- **Week 1:** Send Volume 9 to Shah Luxury. Form legal entity. Open Stripe account.
- **Week 2:** Draft Terms of Service, Privacy, Returns. Trademark search + filing. Domain + DNS.
- **Week 3:** Bank account + Stripe payouts configured. Resend + Klaviyo accounts. Migrate dev DB to Turso production.
- **Week 4:** Shah Luxury Volume 9 responses received. Catalog pricing populated. Initial photography commissioned.

Days 31–60 — Production-Hardening & Soft Launch

- **Week 5:** Schema migration to add Order extensions (subtotal, tax, carrier, tracking). OrderEvent table live. Audit log live.
- **Week 6:** Stripe Tax enabled. Klaviyo flows built (welcome, abandoned cart, post-purchase). Customer account pages complete.
- **Week 7:** Reviews integration. Structured data (JSON-LD) added to product pages. OG images dynamic. Sitemap auto-generated.
- **Week 8:** Soft launch to friends & family — 10 test orders end-to-end. Fulfillment runbook tested with Shah. Concierge inbox staffed.

Days 61–90 — Public Launch & Iterate

- **Week 9:** Accessibility audit + remediation. Lighthouse CI gates in CI. Backup & disaster-recovery drill.
- **Week 10:** Public launch. Press & influencer seeding. Branded paid search live. Klaviyo welcome series active.
- **Week 11:** First post-launch retrospective. Customer feedback session. Refine concierge SLAs.
- **Week 12:** First editorial Journal article published. Klaviyo abandoned-cart performance review. Plan month 4–6 (Volume 8 scaling).

10.3 Ongoing operating cadence

Cadence	Activity	Owner
Daily	Order processing; concierge inbox; Sentry triage	Operations
Daily	Bank settlement reconciliation (Stripe payouts vs ledger)	Finance
Weekly	Marketing email send; Klaviyo performance review	Marketing
Weekly	Sales & fulfillment dashboard review	Operations + Owner
Weekly	Inventory check with Shah	Operations
Monthly	Financial close; P&L review	Finance
Monthly	Journal article published; SEO performance review	Marketing
Monthly	Dependency upgrade (Dependabot PRs merged)	Engineering
Monthly	Customer NPS survey (post-delivery email)	Operations
Quarterly	Secret rotation (JWT, encryption keys, Stripe restricted keys)	Engineering
Quarterly	Performance audit (Lighthouse, bundle analysis)	Engineering
Quarterly	Legal & policy review (terms, privacy)	Owner
Quarterly	Pricing review with Shah	Owner
Annually	Accessibility audit (third-party)	Owner
Annually	Penetration test (recommended at >\$500K ARR)	Engineering
Annually	Insurance policy renewal & review	Owner
Annually	Trademark renewal	Owner

10.4 Hiring plan

Year 1 runs lean: owner-operator + part-time concierge contractor + monthly engineering retainer. Adding roles is triggered by revenue and operational pain, not by aspiration.

Role	Timing	Cost	Triggered by
Owner-operator	Day 1	—	Always-on
Concierge contractor	Month 3	\$1,500–3,000/mo	>20 orders/mo OR >30 emails/wk
Engineering retainer	Month 1	\$2,500–6,000/mo	Always-on; for code, deploys, incidents
Photo retoucher (freelance)	Per project	\$50–100/hr	Per launch
Marketing freelancer	Month 6	\$2,000–4,000/mo	>\$30K MRR
Full-time concierge	Year 2	\$45K–60K	>100 orders/mo
Operations manager	Year 2	\$60K–80K	Multi-vendor; pack/ship in-house

10.5 Key performance indicators

Single-screen weekly dashboard. Owned by the owner. Reviewed every Monday morning. Any metric in amber territory requires a written one-paragraph plan in the next week's meeting.

Metric	Green	Amber	Red
Site uptime	≥ 99.9%	99.5–99.9%	< 99.5%
LCP (mobile, p75)	≤ 2.5s	2.5–4.0s	> 4.0s
Order error rate	< 0.5%	0.5–2%	> 2%
Concierge first-response time	≤ 4h business	4–12h	> 12h
Refund rate	< 3%	3–8%	> 8%
Stripe Radar review rate	< 2%	2–5%	> 5%
Email deliverability (inbox)	≥ 95%	85–95%	< 85%
Cart abandonment rate	< 65%	65–75%	> 75%
NPS (rolling 30d)	≥ 60	30–60	< 30
Repeat purchase rate (12mo)	≥ 15%	5–15%	< 5%

10.6 Backup & disaster recovery

- **Code** — GitHub is the source of truth; daily Vercel deploys preserve history.
- **Database** — Turso point-in-time recovery (24h on Free, 30 days on Scaler). Plus a daily `turso db shell --dump` uploaded to a separate cloud storage bucket (Cloudflare R2, S3).
- **Customer media** — Vercel static assets; rebuilt from repo on deploy.

- **Secrets** — exported quarterly to encrypted offline backup (1Password vault).
- **Recovery drill** — quarterly, restore the most recent dump to a staging database and verify checkout works.
- **RTO (recovery time objective)** — 4 hours for full restore.
- **RPO (recovery point objective)** — 24 hours maximum data loss.

APPENDIX A

Full Prisma Schema

The recommended production schema. Includes everything currently in `prisma/schema.prisma` plus the additions specified across Volumes 2 and 7. Each model is annotated with a short purpose comment.

A.1 Generator & datasource

```
// schema.prisma

generator client {
  provider      = "prisma-client-js"
  previewFeatures = ["driverAdapters"]
}

datasource db {
  provider = "sqlite"
  url      = env("DATABASE_URL")
  // Production: libSQL via @libsql/client driver adapter (Turso)
}
```

A.2 Identity & auth

```
model User {
  id            String    @id @default(cuid())
  email         String    @unique
  emailVerifiedAt DateTime?
  passwordHash  String
  role          String    @default("customer") // customer | admin | super_admin
  failedLogins  Int       @default(0)
  lockedUntil   DateTime?
  twoFactorSecret String? // TOTP secret, encrypted
  twoFactorBackup String? // JSON array of one-time backup codes, encrypted
  marketingConsent Boolean @default(false)
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt

  addresses      Address[]
  orders         Order[]
  sessions       Session[]
  refreshTokens  RefreshToken[]
  wishlists      Wishlist[]
  savedConfigs   SavedConfiguration[]
  reviews       ProductReview[]
  auditLog       AuditLog[]

  @@index([email])
}

model Session {
  id          String    @id @default(cuid())
  userId      String
  token       String    @unique // sha256 hash
  userAgent   String?
  ipAddress   String?
```

```

    expiresAt DateTime
    createdAt DateTime @default(now())
    user      User      @relation(fields: [userId], references: [id], onDelete: Cascade)

    @@index([userId])
    @@index([expiresAt])
}

model RefreshToken {
  id          String    @id @default(cuid())
  userId      String
  tokenHash   String    @unique // sha256
  familyId    String    // for reuse-detection rotation
  used        Boolean   @default(false)
  expiresAt   DateTime
  createdAt   DateTime @default(now())
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([familyId])
  @@index([expiresAt])
}

model PasswordReset {
  id          String    @id @default(cuid())
  email       String
  tokenHash   String    @unique
  used        Boolean   @default(false)
  expiresAt   DateTime
  createdAt   DateTime @default(now())

  @@index([email])
  @@index([expiresAt])
}

```

A.3 Address book (encrypted)

```

model Address {
  id          String    @id @default(cuid())
  userId      String
  label       String?   // 'Home', 'Office'
  fullName    String
  addressLine1 String    // encrypted (AES-256-GCM)
  addressLine2 String?   // encrypted
  city        String
  region      String
  postalCode   String    // encrypted (PII concern in some jurisdictions)
  country     String    // ISO-3166-1 alpha-2
  phone       String?   // encrypted
  isDefault    Boolean   @default(false)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  shippingForOrders Order[] @relation("ShippingAddress")
  billingForOrders Order[] @relation("BillingAddress")

  @@index([userId])
}

```

A.4 Catalog (lightweight; bulk lives in code)

// Note: variant pricing/options live in src/lib/catalog.ts and src/lib/bespoke/*.
 // Database catalog is intentionally minimal – only what admin can edit at runtime.

```
model Product {
  id          String    @id // matches catalog slug: 'sparkle-studs-silver'
  name        String
  collection  String    // 'sparkle-studs' | 'paddle-pave' | 'sparkle-drops' | ...
  family      String    // 'bespoke' | 'configurator'
  wearType    String    // 'earring' | 'pendant' | 'necklace' | 'paddle'
  basePriceCents Int
  active      Boolean   @default(true)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  reviews    ProductReview[]
}
```

A.5 Orders & lifecycle

```
model Order {
  id          String    @id @default(cuid())
  number      String    @unique // 'ORD-1234' human-friendly
  userId      String?
  email       String    // captured even for guest checkout
  status      String    // see Appendix D state machine
  paymentIntentId String? @unique
  checkoutSessionId String? @unique
  // money
  currency    String    @default("USD")
  subtotalCents Int
  shippingCents Int      @default(0)
  taxCents    Int      @default(0)
  discountCents Int      @default(0)
  totalCents  Int
  // shipping
  shippingAddressId String?
  billingAddressId  String?
  carrier           String?
  trackingNumber    String?
  trackingUrl       String?
  // fulfillment timestamps
  paidAt           DateTime?
  productionStartedAt DateTime?
  qualityCheckedAt DateTime?
  shippedAt        DateTime?
  deliveredAt      DateTime?
  completedAt      DateTime?
  cancelledAt      DateTime?
  cancelReason     String?
  // notes
  internalNotes    String? // admin-only
  customerNote     String? // gift message, special request
  createdAt        DateTime @default(now())
  updatedAt        DateTime @updatedAt

  user          User?    @relation(fields: [userId], references: [id])
  shippingAddress Address? @relation("ShippingAddress", fields: [shippingAddressId], references: [id])
  billingAddress Address? @relation("BillingAddress", fields: [billingAddressId], references: [id])
}
```

```

    items            OrderItem[]
    events            OrderEvent[]
    returns           ReturnRequest[]
    couponRedemptions CouponRedemption[]

    @@index([userId, createdAt])
    @@index([status, createdAt])
    @@index([email])
}

model OrderItem {
  id          String   @id @default(cuid())
  orderId     String
  productId   String   // composite key from catalog (e.g. 'sparkle-studs-silver__tw-050')
  productName String   // snapshot
  variantKey  String?  // selection map snapshot
  quantity    Int
  unitPriceCents Int    // snapshot at order time
  metadata    String?  // JSON of selections (metal, stone, treatment, size, engraving)
  order       Order    @relation(fields: [orderId], references: [id], onDelete: Cascade)

  @@index([orderId])
  @@index([productId])
}

model OrderEvent {
  id          String   @id @default(cuid())
  orderId     String
  type        String   // 'status_changed' | 'note_added' | 'email_sent' | 'webhook_received'
  fromStatus  String?
  toStatus    String?
  actorId     String?  // userId of admin who triggered; null for system
  metadata    Json?
  createdAt   DateTime @default(now())
  order       Order    @relation(fields: [orderId], references: [id], onDelete: Cascade)

  @@index([orderId, createdAt])
}

```

A.6 Returns, coupons, wishlist, reviews, subscriptions

```

model ReturnRequest {
  id          String   @id @default(cuid())
  orderId     String
  rmaCode     String   @unique
  reason      String   // 'changed_mind' | 'defective' | 'wrong_item' | 'sizing'
  status      String   // 'requested' | 'approved' | 'received' | 'refunded' | 'rejected'
  refundCents Int?
  customerNote String?
  internalNote String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
  order       Order    @relation(fields: [orderId], references: [id])

  @@index([orderId])
  @@index([status])
}

model Coupon {
  id          String   @id @default(cuid())

```

```

    code          String    @unique
    description    String?
    discountType   String    // 'percent' | 'amount'
    discountValue  Int       // percent points (10 = 10%) or cents
    minOrderCents  Int?
    maxUses        Int?
    maxUsesPerUser Int       @default(1)
    validFrom      DateTime
    validUntil     DateTime?
    active         Boolean   @default(true)
    createdAt      DateTime  @default(now())

    redemptions    CouponRedemption[]

    @@index([code])
    @@index([active, validUntil])
}

model CouponRedemption {
  id          String    @id @default(cuid())
  couponId    String
  orderId     String
  userId      String?
  appliedCents Int
  createdAt   DateTime  @default(now())
  coupon      Coupon    @relation(fields: [couponId], references: [id])
  order       Order     @relation(fields: [orderId], references: [id])

  @@index([couponId])
  @@index([orderId])
  @@unique([couponId, userId])
}

model Wishlist {
  id          String    @id @default(cuid())
  userId      String
  name        String    @default("Favourites")
  createdAt   DateTime  @default(now())
  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  items       WishlistItem[]

  @@index([userId])
}

model WishlistItem {
  id          String    @id @default(cuid())
  wishlistId  String
  productId   String
  variantKey  String?
  addedAt     DateTime  @default(now())
  wishlist    Wishlist  @relation(fields: [wishlistId], references: [id], onDelete: Cascade)

  @@index([wishlistId])
}

model SavedConfiguration {
  id          String    @id @default(cuid())
  userId      String
  productId   String    // 'sparkle-studs-silver' or 'ball-pendant' configurator
  selections  Json      // serialized selection map
  name        String?
  createdAt   DateTime  @default(now())

```

```

    user      User      @relation(fields: [userId], references: [id], onDelete: Cascade)

    @@index([userId])
}

model ProductReview {
  id          String    @id @default(cuid())
  productId   String
  userId      String?
  orderId     String?
  rating      Int       // 1-5
  title       String?
  body        String
  authorName  String    // displayed publicly; not necessarily equal to user.name
  photos      Json?     // array of CDN URLs
  approved    Boolean   @default(false)
  createdAt   DateTime  @default(now())
  product     Product   @relation(fields: [productId], references: [id])
  user        User?     @relation(fields: [userId], references: [id])

  @@index([productId, approved, createdAt])
  @@index([userId])
}

model EmailSubscription {
  id          String    @id @default(cuid())
  email       String    @unique
  source      String?   // 'footer' | 'checkout' | 'popup'
  consentedAt DateTime  @default(now())
  unsubscribedAt DateTime?
  ipAddress   String?
}

model WebhookEvent {
  id          String    @id // Stripe event id, used as idempotency key
  provider    String    // 'stripe' | 'resend' | 'carrier'
  type        String
  payload     Json
  status      String    @default("pending") // 'pending' | 'processed' | 'failed'
  processedAt DateTime?
  errorMsg    String?
  createdAt   DateTime  @default(now())

  @@index([provider, type])
  @@index([status, createdAt])
}

model AuditLog {
  id          String    @id @default(cuid())
  actorId     String?
  action      String
  resourceType String?
  resourceId  String?
  metadata    Json?
  ipAddress   String?
  userAgent   String?
  createdAt   DateTime  @default(now())
  actor       User?     @relation(fields: [actorId], references: [id])

  @@index([actorId, createdAt])
  @@index([action, createdAt])
  @@index([resourceType, resourceId])
}

```


}

APPENDIX B

API Endpoint Reference

Complete API surface. Every endpoint is a Next.js App Router route handler under `src/app/api/`. Authentication via HTTP-only cookies; CSRF check on mutating requests (Origin = host).

B.1 Auth

Endpoint	Auth	Description
POST /api/auth/register	Public	Create account. body: {email, password}
POST /api/auth/login	Public	Login. body: {email, password}. Sets cookies.
POST /api/auth/logout	Auth	Invalidate session. Clears cookies.
POST /api/auth/refresh	Refresh cookie	Rotate access + refresh tokens.
POST /api/auth/reset	Public	Request password reset email.
POST /api/auth/reset/confirm	Public	Submit new password with token.
POST /api/auth/2fa/enroll	Auth	Begin TOTP enrollment.
POST /api/auth/2fa/verify	Auth	Confirm TOTP code; persist secret.

B.2 Account

Endpoint	Auth	Description
GET /api/account	Auth	Current user + addresses.
PATCH /api/account	Auth	Update name, marketing consent.
POST /api/account/password	Auth	Change password (requires current).
GET /api/account/addresses	Auth	List addresses (decrypted).
POST /api/account/addresses	Auth	Add address.
PATCH /api/account/addresses/[id]	Auth	Update address.
DELETE /api/account/addresses/[id]	Auth	Remove address.
GET /api/account/orders	Auth	List user's orders.
GET /api/account/orders/[id]	Auth	Order detail with items + events.
POST /api/account/dsar	Auth	GDPR data export (rate-limited 1/day).

Endpoint	Auth	Description
POST /api/account/delete	Auth	Right-to-erasure; queues 30-day deletion.

B.3 Catalog & configurator

Endpoint	Auth	Description
GET /api/products	Public	List products (filterable by collection).
GET /api/products/[id]	Public	Product detail.
POST /api/configurator/price	Public	Estimate price for a selection map. body: {productId, selections}
POST /api/configurator/save	Auth	Persist a SavedConfiguration. body: {productId, selections, name?}
GET /api/configurator/saved	Auth	List saved configurations.
DELETE /api/configurator/saved/[id]	Auth	Remove a saved configuration.

B.4 Cart, wishlist & checkout

Endpoint	Auth	Description
POST /api/checkout	Public	Create Stripe Checkout session. body: {items, shippingAddress, couponCode?}. Server recomputes all prices.
POST /api/webhook/stripe	Stripe-signed	Receives checkout.session.completed; idempotent via WebhookEvent.
GET /api/wishlist	Auth	Current wishlist.
POST /api/wishlist	Auth	Add item.
DELETE /api/wishlist/[id]	Auth	Remove item.

B.5 Returns & reviews

Endpoint	Auth	Description
POST /api/returns	Auth	Request return. body: {orderId, reason, note}
GET /api/returns/[id]	Auth	Return status.
GET /api/reviews/product/[id]	Public	Reviews for a product.
POST /api/reviews	Auth	Submit review (requires past delivered order for that product).

B.6 Marketing & contact

Endpoint	Auth	Description
POST /api/newsletter	Public	Subscribe email. Honors GPC signal.
POST /api/newsletter/unsubscribe	Public	One-click unsubscribe; honors RFC 8058.
POST /api/contact	Public	Concierge contact form. Rate-limited.

B.7 Admin

Endpoint	Auth	Description
GET /api/admin/orders	Admin	List all orders (paginated).
PATCH /api/admin/orders/[id]/status	Admin	Move order through state machine; writes OrderEvent.
POST /api/admin/orders/[id]/refund	Admin	Trigger Stripe refund + status transition.
POST /api/admin/orders/[id]/tracking	Admin	Set carrier + tracking; triggers shipped email.
GET /api/admin/users	Admin	User search.
PATCH /api/admin/users/[id]	Super admin	Change role; clear logout.
GET /api/admin/coupons	Admin	List coupons.
POST /api/admin/coupons	Admin	Create coupon.
PATCH /api/admin/coupons/[id]	Admin	Deactivate coupon.
GET /api/admin/audit-log	Admin	Query audit log.

B.8 Internal

Endpoint	Auth	Description
GET /api/health	Public	Liveness probe: returns 200 if app responds; checked by uptime monitor.
GET /api/cron/sitemap	Cron secret	Daily sitemap regeneration; called by Vercel cron.
GET /api/cron/cleanup	Cron secret	Daily expired session + token + WebhookEvent cleanup.

APPENDIX C

Environment Variables Checklist

Every variable required for production. **Required** means the app will not function correctly without it; **Recommended** means a feature degrades but the site still works.

C.1 Required

Variable	Owner	Source / how to obtain
DATABASE_URL	Engineering	Turso console → libsql://[db].turso.io
DATABASE_AUTH_TOKEN	Engineering	Turso console → database settings
JWT_ACCESS_SECRET	Engineering	Generate: openssl rand -hex 64
FIELD_ENCRYPTION_KEY	Engineering	Generate: openssl rand -hex 32
STRIPE_SECRET_KEY	Finance / Owner	Stripe dashboard → Developers → API keys
STRIPE_PUBLISHABLE_KEY	Finance / Owner	Stripe dashboard
STRIPE_WEBHOOK_SECRET	Engineering	Stripe dashboard → Webhooks → endpoint → signing secret
NEXT_PUBLIC_SITE_URL	Owner	Your primary domain (https://yourdomain.com)
RESEND_API_KEY	Engineering	Resend dashboard → API keys
FROM_EMAIL	Owner	concierge@yourdomain.com (must be DKIM-verified)
NODE_ENV	Engineering	production

C.2 Recommended

Variable	What breaks if missing
SENTRY_DSN	Error monitoring offline
UPSTASH_REDIS_REST_URL	Rate limiting falls back to in-process (per-instance)
UPSTASH_REDIS_REST_TOKEN	Same
KLAVIYO_PRIVATE_API_KEY	Marketing email automation offline
KLAVIYO_PUBLIC_API_KEY	Klaviyo profile sync offline
PLAUSIBLE_DOMAIN	Analytics offline

Variable	What breaks if missing
NEXT_PUBLIC_GA_ID	Google Analytics 4 offline
ADMIN_IP_ALLOWLIST	Admin endpoints open to all IPs
CRON_SECRET	Vercel cron endpoints unauthenticated
TURNSTILE_SITE_KEY / TURNSTILE_SECRET_KEY	Public-form bot protection off

C.3 Local dev only

Stored in `.env.local`. NEVER committed. `.env.example` in the repo contains placeholder values to seed a new developer's setup.

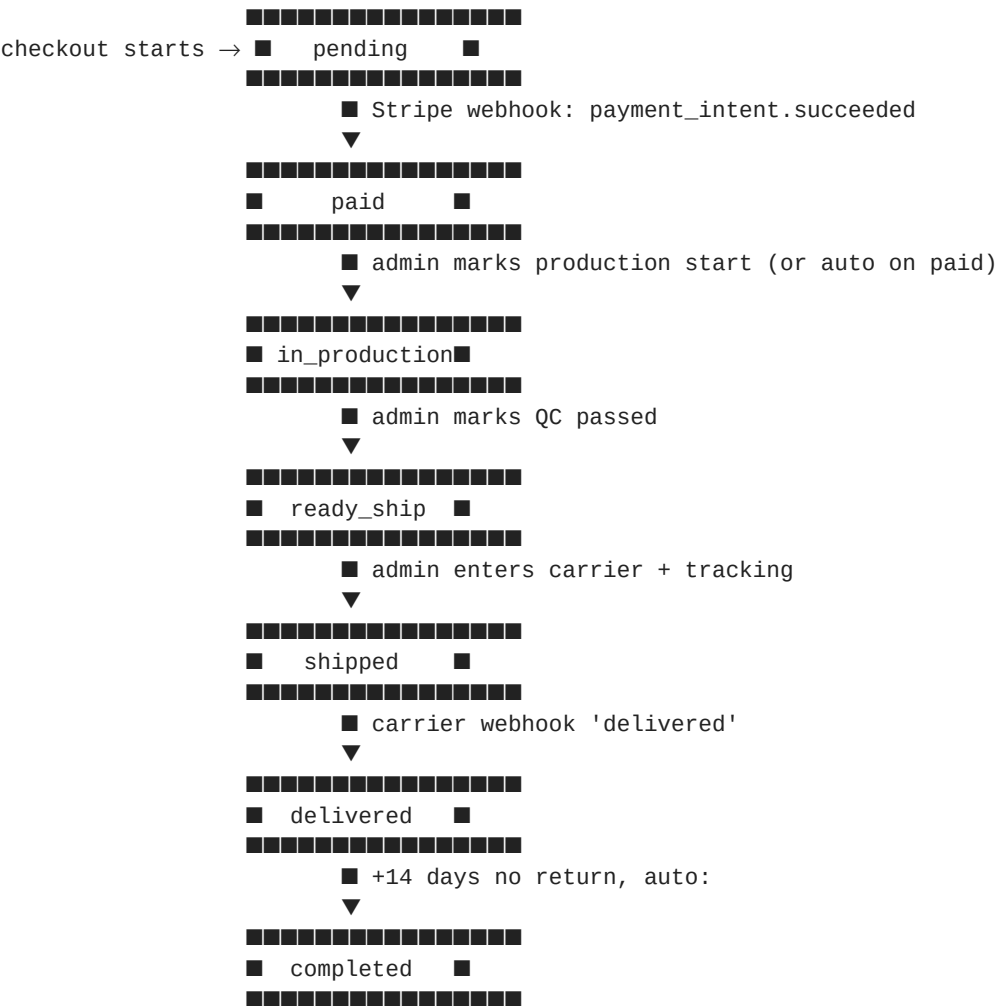
C.4 Secret rotation cadence

Secret	Rotate	Process
JWT_ACCESS_SECRET	Quarterly	Existing tokens invalidated; users sign back in
FIELD_ENCRYPTION_KEY	Annually or on incident	Run re-encrypt script on Address rows
STRIPE_SECRET_KEY	On incident or staff change	Roll in Stripe dashboard
STRIPE_WEBHOOK_SECRET	On endpoint change	Update endpoint in Stripe; redeploy
RESEND_API_KEY	Annually or on incident	Create new key, swap, delete old
KLAVIYO keys	Annually or on incident	Same

APPENDIX D

Order Status State Machine

The Order.status field follows this state machine. Transitions are enforced server-side; an admin cannot, e.g., move a pending order directly to shipped without passing through paid.



- Branches at any stage:
- pending → cancelled (customer abandons / admin cancels)
 - paid → cancelled (admin pre-production cancel; full refund)
 - in_production → cancelled (admin cancel; partial or full refund)
 - delivered → return_requested → return_received → refunded
 - delivered → exchanged (return + remake)

D.1 Transition matrix

From	To	Triggered by
pending	paid	Stripe checkout.session.completed webhook
pending	cancelled	Customer abandons (24h timeout) or admin cancels

From	To	Triggered by
paid	in_production	Admin or auto-transition on paid
paid	cancelled	Admin cancels with full refund
in_production	ready_ship	Admin marks QC complete
in_production	cancelled	Admin cancels; refund net of work completed
ready_ship	shipped	Admin enters tracking number; triggers email
shipped	delivered	Carrier webhook 'delivered' event
shipped	lost_in_transit	Carrier webhook 'lost' or manual after 30d
delivered	completed	Auto: +14 days, no return request
delivered	return_requested	Customer initiates return
return_requested	return_received	Admin confirms package received
return_received	refunded	Admin issues refund
return_received	exchanged	Admin issues remake order

D.2 Email triggers per transition

Transition	Email sent to customer
* → paid	Order received (with invoice + lead time)
* → in_production	Your piece is being made
* → ready_ship	Quality check passed
* → shipped	On the way + tracking link
* → delivered	Delivered + care guide
* → completed	(no email — internal only)
* → cancelled	Order cancelled (with refund detail)
* → refunded	Refund processed

APPENDIX E

Email Template Library

Every transactional email follows a shared layout (header logo, body, footer with unsubscribe). Marketing emails additionally include preference link and physical address (CAN-SPAM).

E.1 Layout principles

- Single-column 600px maximum width; responsive at narrower viewports.
- Serif headline + sans-serif body — matches site typography (Times Roman + Helvetica equivalents in email: Georgia + Arial fallback).
- Real text, not images, for the body. Images are decorative only.
- Plain-text alternate generated for every HTML email (Resend handles this automatically).
- Footer: unsubscribe link, physical business address (CAN-SPAM), 'View in browser' link.
- No attachments (use links instead). Receipts are linked from the order page.

E.2 Order received — subject & body

Subject: Your [BRAND] order – #{order.number}

Preview: Thank you for choosing [BRAND]. Here's what comes next.

[BRAND]

Dear {first_name},

Thank you for your order. We're honoured to be part of your story.

YOUR ORDER · #{order.number}

[illegible]

```
Subtotal: {subtotal}
Shipping: {shipping}
    Tax: {tax}
    Total: {total}
```

WHAT HAPPENS NEXT

1. Your piece begins production within 48 hours.
2. Estimated completion: {lead_time_days} business days.
3. Once ready, we'll quality-check, photograph, and ship.
4. You'll receive tracking when your piece is on the way.

QUESTIONS?

Your concierge is {concierge_name}. Reply to this email or call us at {concierge_phone} – anytime.

View order: {order_url}

- The [BRAND] team

[BRAND] · {business_address}

You received this because you placed an order.

This is not a marketing email.

E.3 Shipped notification

Subject: Your [BRAND] order is on the way

Dear {first_name},

Your piece has left the atelier.

```
Tracking number: {tracking_number}
Carrier:         {carrier}
Estimated arrival: {eta}
```

```
Track shipment: {tracking_url}
```

A few notes for the moment your package arrives:

- The outer box is intentionally discreet for security.
- Inside, you'll find a sealed inner box, a polishing cloth, your certificate of authenticity, and a care card.
- If anything is not as it should be, reply to this email within 14 days and we'll make it right.

- The [BRAND] team

E.4 Templates to author (full list)

- Order received · Order cancelled · Refund issued
- Production started · QC passed · Shipped · Delivered
- Return requested · Return received · Refund or exchange confirmed
- Password reset · Password changed confirmation · New device sign-in alert
- Welcome (newsletter) · Welcome series #2 (founder note) · Welcome series #3 (concierge intro)
- Abandoned cart (1h, 24h, 72h)
- Post-purchase nurture (+7d care, +30d styling, +90d review request)
- Birthday (no discount) · VIP first-look · Win-back

APPENDIX F

Vendor & Tooling Comparison Matrix

Used when evaluating alternates. Recommended choice highlighted in **bold**.

F.1 Database

Vendor	Model	Year-1 cost	Notes
Turso (libSQL)	SQLite, edge-replicated	\$0	Free covers projected year-1 traffic; current choice
Neon	Postgres serverless	\$0–\$19	More mature ecosystem; cold starts a concern
Planetscale	MySQL	\$39+	Strong tooling; minimum \$39/mo
Supabase	Postgres + auth + storage	\$0–\$25	Bundles auth; vendor lock-in higher
AWS RDS	Postgres / MySQL	\$50–\$200+	Ops overhead; reserve for scale

F.2 Hosting

Vendor	Year-1 cost	Notes
Vercel	\$20–\$40/mo	Best-in-class Next.js; recommended
Netlify	\$0–\$19/mo	Capable; weaker Next.js parity
Cloudflare Pages	\$0–\$5/mo	Excellent value; weaker Server Components support
AWS Amplify	\$0–\$30/mo	More complex; rarely worth it for solo operator
Self-hosted on Hetzner / Fly	\$10–\$40/mo	Cheapest; ops cost is your time

F.3 Email — transactional

Vendor	Free tier	Paid	Notes
Resend	3K/mo	\$20+	Modern DX; React-email integration
Postmark	100/mo	\$15+	Top deliverability reputation
Mailgun	100/day	\$15+	More features; less polished DX
AWS SES	Generous	\$0.10/1K	Cheapest; ops cost is your time

F.4 Email — marketing

Vendor	Free tier	Paid	Notes
Klaviyo	500 contacts	\$45+	E-commerce flow primitives; strongest for jewelry DTC
Mailchimp	500 contacts	\$13+	Cheaper; weaker e-commerce
Customer.io	—	\$100+	Stronger for SaaS than e-commerce
ConvertKit	1K contacts	\$15+	Creator-focused; weaker e-commerce

F.5 Payments

Vendor	US card	Intl card	Notes
Stripe	2.9% + \$0.30	+1.5%	De facto standard; best DX; Stripe Tax + Radar
Adyen	Negotiated	Negotiated	Better at scale (\$1M+/mo); over-engineered for v1
Braintree (PayPal)	2.59% + \$0.49	+1%	Cheaper; weaker DX; consider for PayPal-heavy markets

F.6 Reviews

Vendor	Free tier	Paid	Notes
Junip	Yes	\$29+	Cheapest with photo reviews + Klaviyo integration
Okendo	—	\$29+	Stronger UGC features; brand-leaning
Yotpo	Yes (limited)	\$15+	Larger ecosystem; enterprise-leaning
Trustpilot	Limited	\$200+	More reviews credibility; expensive

APPENDIX G

Glossary

Terms used throughout this document, in plain language.

App Router

Next.js routing system based on directories under `app/`. Each folder becomes a route; `page.tsx` is the page component.

Argon2id

Modern memory-hard password hashing function. Resists GPU-based brute-force better than `bcrypt` for the same compute budget.

AOV — Average Order Value

Total revenue divided by number of orders. For [BRAND], projected \$1,500–\$2,500.

CCPA

California Consumer Privacy Act. Grants California residents rights similar to GDPR.

CDN — Content Delivery Network

Geographically distributed network of edge servers that cache static assets close to users.

Configurator

The customize page (e.g. `/customize/sparkle-studs-silver`) where customers select variations (metal, stone, size, treatment) before purchase.

Core Web Vitals

Google's UX metric set: LCP (load), INP (interactivity), CLS (layout stability). Used for SEO ranking.

CSP — Content Security Policy

HTTP header restricting which scripts, styles, images can load. Defense against XSS.

CSRF — Cross-Site Request Forgery

Attack where a malicious site tricks the user's browser into making a state-changing request to [BRAND]. Mitigated by Origin/Host check on mutating routes.

DDP / DDU

Delivered Duty Paid / Unpaid. Whether the seller (DDP) or buyer (DDU) handles import duties.

DSAR — Data Subject Access Request

GDPR-mandated user right to download a copy of all data held about them. Must be fulfilled within 30 days.

DTC — Direct-to-Consumer

Selling directly without a retail middleman. The [BRAND] business model.

Field encryption

AES-256-GCM encryption of specific database columns (addresses, phone numbers). Keys held in env vars; encrypted blobs stored in DB.

GDPR

General Data Protection Regulation. EU privacy law; applies to any business serving EU customers.

Idempotency

A property where repeated calls produce the same result. Critical for webhooks where Stripe may retry.

JWT — JSON Web Token

Signed token format used for access tokens. Stored in HTTP-only cookies.

Klaviyo flow

An automated email sequence triggered by an event (signup, abandoned cart, etc.).

LCP — Largest Contentful Paint

Time until the largest visible element renders. Target $\leq 2.5s$.

Lighthouse

Google's performance / accessibility / SEO audit tool. Run in CI.

libSQL / Turso

Edge-replicated SQLite fork; chosen database for [BRAND].

Middleware

Next.js code (`middleware.ts`) that runs before every request — used for auth, CSRF, rate limit, redirects.

Order state machine

Set of allowed transitions through order statuses. See Appendix D.

Pavé

Setting style where small stones are set so closely that the metal disappears, creating a paved-with-stones surface. Used in [BRAND]'s diamond-set pieces.

Prisma

TypeScript ORM used to query the database. Schema in `prisma/schema.prisma`.

Rate limiting

Cap on requests per IP / per user per time window. Defense against brute-force, scraping, fraud.

Refresh token rotation

Each time the refresh token is used, a new one is issued and the old one marked used. Reuse triggers family-wide revocation.

Resend

Transactional email provider chosen for [BRAND].

Server Component

React component that runs on the server only; can fetch data directly; no JS shipped to client. Next.js App Router default.

Server function

Code that runs on Vercel's serverless infrastructure — API routes, page renders, etc.

Shah Luxury

The brand partner / manufacturer supplying [BRAND]'s pieces.

SKU — Stock Keeping Unit

Unique product identifier. Format: collection-metal-stone-size.

Stripe Checkout

Stripe-hosted payment page. Customer data and card never touch [BRAND]'s server (SAQ-A PCI scope).

Stripe webhook

HTTP callback from Stripe to `/api/webhook/stripe` when an event (`payment_intent.succeeded`, etc.) occurs.

TOTP

Time-based one-time password. Standard for app-based 2FA (Google Authenticator, 1Password, etc.).

Vercel

Hosting platform for the [BRAND] site. Owns Next.js.

WAF — Web Application Firewall

Edge filter that blocks attacks (SQL injection, bot traffic) before they reach the app.

End of document. v1.0 — please file any corrections to engineering for inclusion in v1.1.